

K Shortest and Dissimilar Paths

May 1, 2026

CS/ECE/ISyE 524 — Introduction to Optimization — Spring 2026

K Shortest and Dissimilar Paths

Caleb Taylor (ctaylor49@wisc.edu), Corinne Orton (corton@wisc.edu), Thomas Przybylinski (tprzybylinsk@wisc.edu)

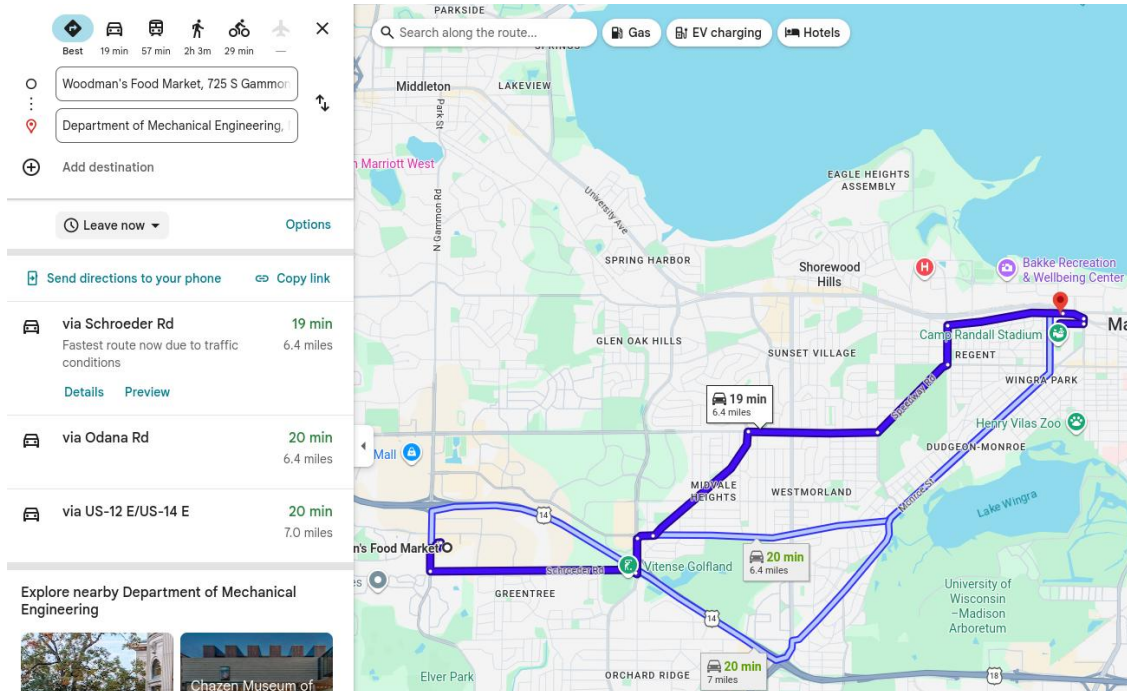
Table of Contents

1. Introduction
2. Mathematical Model
3. Implementation and Solutions
4. Discussion of Results
5. Excursions
6. Conclusion

1. Introduction

Our project is finding k shortest and dissimilar paths between two nodes within a graph. This has been called the K Dissimilar Paths problem [1]. In many places, we also call these paths “diverse”. There are many cases where we may want to find more than one shortest path between two nodes in a network, one example being travel routing. For example, when using Google Maps, a user is typically shown multiple distinct quick routes that take a similar amount of time. This is helpful in case the user wants to avoid highways, needs to make a stop on the way, or prefers one route to another for any reason. Seeing multiple options allows them to choose the route that works best for them.

Many algorithms that have been created to solve this problem [2], and we have chosen to use a penalty-based method that uses a combination of models that iteratively create diverse paths.



Network

The graph network we have chosen is a grid structure where intersections and locations are represented by a node, and roads or other routes are represented by an arc between them. Each node has an edge to the node directly up, down, left, and right from it (so there are no diagonal connections.) Each edge has a “cost” associated with it. As a simplifying assumption, we assume this cost contains all the information needed to route the quickest path. So we could think of this cost as the time it takes to go from one node to another, but could also represent other desirable criteria such as riskiness of traffic jams, roads people don’t like driving on, etc. Hence, for us the “shortest” path as the one with the lowest cost. As a simplifying assumption, we do not take into account any changes to this cost over time.

We chose a grid pattern because these occur in streets layouts around the world and because it provides interesting complications by providing many reasonable paths between two nodes. If the graph had only a few reasonable paths between nodes, such as if we were modeling city-to-city drives, then using the common method of simply preventing edge reuse would probably be sufficient. However, for large grids using this constraint creates paths that look generally similar going in the same general direction so we need to be more thoughtful.

Dissimilarity

Thus, to ensure the k paths are not too close together, we introduce a “diversity cost” for each edge. To illustrate the idea behind this, imagine there are two paths and we are currently on one of them. If there are many places where I can make small adjustments to the directions I’m following to arrive at the other path and continue on, then it’s hard to say they are dissimilar paths.

More specifically, at each point in a path we measure the fewest number of nodes we have to “hop” through to get to a different path and generate a cost based on the number of hops. The fewer the hops, the larger the cost. These costs are summed for every edge. Note that this dissimilarity is

agnostic to the shortest-path cost because we are trying to measure structural differences, somewhat analagous to changes in directions rather than travel time.

One can also think of this as a generalization of a common way of finding diverse paths: minimizing common edges. Since common edges have 0 hops, these edges are maximally penalized, but then we also penalize edges that are close to existing edges. One issue common issue for dissimilar path models is that they can create seeming diversity by adding small unnecessary detours[2], and this criteria also helps prevent this from happening.

We set the diversity cost equal to $\frac{1}{\sqrt{d+1}}$, where d is the minimum number of hops from the edge to an edge in an established path (i.e. the number of nodes between it and the closest established edge.). Since diversity cost is nonlinear, costs are not additive if an edge is close to two existing paths.

Finding The Paths

To find diverse paths, we first find one shortest path, then find another shortest path distinct from it, then another distinct from both, etc. We call the shortest paths we have already found “established paths.” We define the diversity of an edge in terms of the number of nodes the edge must “hop” across to reach an edge of an established path.

This is appropriate because our simplifying assumption is that we just need to provide reasonable alternative paths for users. So finding the best diverse path given a shortest path, or a set of diverse paths is sufficient and we will see this also provides us with a relatively fast model.

Tradeoff

A path is not a useful alternative just because it is dissimilar. Visiting every city in the world before going to your bedroom from your living room is extremely dissimilar to paths that exist entirely within your house, but that’s not a path anyone will use. So there is inherently a tradeoff between dissimilarity and cost. Instead of trying to find this middle ground with reasonable constraints, we will model this problem as a tradeoff between paths being short and being dissimilar. Someone could then tune the tradeoff to suit their needs.

2. Mathematical model

2.1 Baseline Model Formulation

We solve a sequence of shortest-path linear programs. First a standard shortest path problem, then additional shortest path problems with diversity penalties based on previously used edges. For the shortest path problem we follow the notation of [3] (i.e., the IYSE 728 textbook).

Let $G = (V, A)$ be a directed graph. For each node $v \in V$, let $\delta^{\text{out}}(v)$ be the set of arcs leaving node v and let $\delta^{\text{in}}(v)$ be the set of incoming arcs to v . That is,

$$\delta^{\text{out}}(v) = \{vw \in A : w \in V\} \tag{1}$$

$$\delta^{\text{in}}(v) = \{uv \in A : u \in V\} \tag{2}$$

Assume $s \in V$ is the source node and $t \in V$ is the sink node. Define the supply-vector b_v for each

$v \in V$ as

$$b_v = \begin{cases} 1 & \text{if } v = s, \\ -1 & \text{if } v = t, \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

Assume it cost c_a to use arc $a \in A$. Let $x_a \geq 0$ for all $a \in A$ be the decision variables that indicate whether flow travels across arc a from source s to sink t . We now give the baseline linear program formulation.

Initial Shortest Path (ISP) LP The first path is obtained by solving a standard shortest path model.

$$\min \sum_{a \in A} c_a x_a \quad (4)$$

$$\text{s.t.} \quad \sum_{a \in \delta^{\text{out}}(v)} x_a - \sum_{a \in \delta^{\text{in}}(v)} x_a = b_v \quad \text{for all } v \in V, \quad (5)$$

$$x_a \geq 0 \text{ for all } a \in A. \quad (6)$$

Because our formulation is a standard shortest path problem, the optimal solution will be an integer solution due to the incidence matrix being totally unimodular. Let the arcs utilized by the solution to the initial shortest path LP be $P^{(1)} = \{a \in A : x_a > 0\}$. Let $U^{(1)} = P^{(1)}$, where $U^{(1)}$ denotes the set of used arcs in the first shortest path.

Hops-To-Used-Arcs (DTUA) LP We next solve an auxiliary linear program that determines the number of steps (i.e., the number of hops) each arc $a \in A$ is from the set of used arcs $U^{(1)}$ in part (1.). This is meant to model the idea that if a user only has to make a few adjustments to their path to arrive at another path, then those two paths are likely not too different from each other. For example, if we have the arcs $(u, v) \in A$ and $(v, w) \in A$, then moving from (u, v) to (v, w) is one step. Before we give the LP of step (2.), define the set of adjacent arcs

$$\mathcal{S} = \{((u, v), (v, w)) \in A \times A : u, v, w \in V\}.$$

where $((u, v), (v, w)) \in \mathcal{S}$ means that arc (u, v) directly precedes arc (v, w) . We now give the LP of step (2).

$$\max \sum_{a \in A} z_a \quad (7)$$

$$\text{s.t.} \quad z_a = 0 \text{ for all } a \in U^{(1)}, \quad (8)$$

$$z_{(v, w)} \leq z_{(u, v)} + 1 \text{ for all } (v, w) \in A \setminus U^{(1)}, (u, v) \in A, \text{ such that } ((u, v), (v, w)) \in \mathcal{S}, \quad (9)$$

$$z_a \geq 0 \text{ for all } a \in A. \quad (10)$$

The solution to the DTUA LP is the minimum number edge-to-edge hops or steps needed to reach arc a from a previously used arc. To make sense of the second to last constraint, if $z_{(u, v)}$ is the number of arc-to-arc steps from the used set $U^{(1)}$ to arc (u, v) and we know (v, w) comes immediately after (u, v) , then (v, w) cannot be more than one additional step farther away from $U^{(1)}$ than (u, v) is. Also, the second to last constraint is written as an upper bound, but because the LP is maximizing, each z_a gets pushed as large as possible while still respecting all other inequalities.

Diverse Penalty Construction Once the arc-to-used set distance values z_a are computed in (2), we calculate the diversity penalty d_a , defined as, for $a \in U^{(1)}$:

$$d_a = \frac{1}{\sqrt{z_a + 1}}$$

and $d_a = 0$ for $a \notin U^{(1)}$.

We chose this penalty so that it is maximized for shared edges, and decreases over distance. To alleviate a concern that a simple $\frac{1}{z_a + 1}$ would decrease too rapidly, we take the square root of the denominator to slow the decrease down. In testing, this worked well enough.

Diversified Shortest Path (DSP) LP In this step we solve a similar LP to the initial shortest path LP, but we find a second shortest path with a modified objective. The objective function of the DSP LP carries a tradeoff weight λ for the sum of the diversity penalties of the arcs chosen. The parameter λ controls the tradeoff between two goals: smallest total travel cost c_a and diversity from earlier paths d_a . This allows us to quantify the inherent subjectiveness of diversity and when extra diversity is worth some additional path cost. The LP for step (4) is:

$$\min \quad \sum_{a \in A} c_a x_a + \lambda \sum_{a \in A} d_a x_a \quad (11)$$

$$\text{s.t.} \quad \sum_{a \in \delta^{\text{out}}(v)} x_a - \sum_{a \in \delta^{\text{in}}(v)} x_a = b_v \quad \text{for all } v \in V, \quad (12)$$

$$x_a \geq 0 \text{ for all } a \in A. \quad (13)$$

From this point, we have our second shortest path solution, denoted $P^{(2)}$. Now set $U^{(2)} = U^{(1)} \cup P^{(2)}$. The solution procedure repeats steps (2) through (4). A summary of the solution procedure is below. Note that it does not use the same step-by-step numbering.

Baseline Sequential N-Diverse Shortest Paths Procedure. **Inputs:** Directed graph $G = (V, A)$, source node s , sink node t , desired number of shortest paths N , tradeoff parameter λ , and diversity penalty function d_a .

Step 1: Solve ISP LP and obtain first path $P^{(1)}$. if no feasible solution exists, stop.

Step 2: Set $U^{(1)} = P^{(1)}$.

Step 3: For $k = 2, \dots, N$:

- solve HTUA LP using $U^{(k-1)}$ to obtain latest z_a values.
- solve for arc penalties d_a from $U^{(k-1)}$ using z_a values.
- solve diverse shortest path problem using $U^{(k-1)}$ and tradeoff parameter λ to obtain $P^{(k)}$. If no feasible solution exists, exit loop.
- update $U^{(k)} = U^{(k-1)} \cup P^{(k)}$.
- proceed to the next iteration by replacing k with $k + 1$, unless $k = N$.

Output: collected of shortest paths $\{P^{(1)}, \dots, P^{(N)}\}$, unless the procedure terminated early due to not having N feasible solutions.

3. Implementation and Solutions

Graph Representation

First, we created data structures to represent our graph to help keep things simple and organized down the road.

```
[1]: #If we don't set a cost, set it to 1
      _default = Dict{Symbol,Number}()
      _default[:cost] = 1

      #Setup Nodes. They have an id, and an x/y coordinate for graphing as well as
      ↪properties
      abstract type AbstractNode end

      #Nodes have an id that uniquely represents them, as well as x/y coordinates and
      ↪properties
      struct Node <: AbstractNode
          id::Symbol
          x::Number
          y::Number
          _props::Dict{Symbol,<:Number}

          Node(id::Union{String,Symbol}, x::Number, y::Number) = new(Symbol(id), x,
          ↪y, Dict{Symbol,Number}())
          Node(id::Union{String,Symbol}, x::Number, y::Number, props::Dict{Symbol,<:
          ↪Number}) = new(id, x, y, props)
          Node(id::Union{String,Symbol}, x::Number, y::Number; kwargs...) =
          ↪new(Symbol(id), x, y, Dict(kwargs))
      end

      #This allows us to reference a Node without having to re-create the nodes
      #So, for example, they can be used as keys on the JuMP model
      struct NodeKey <: AbstractNode
          id::Symbol
          NodeKey(id::String) = new(Symbol(id))
          NodeKey(id::Symbol) = new(id)
      end

      #This just enforces that id identifies the node
      Base.hash(x::AbstractNode, y::UInt64) = hash(x.id, y)
      Base.==(x::AbstractNode, y::AbstractNode) = x.id == y.id
      Base.isequal(x::AbstractNode, y::AbstractNode) = isequal(x.id, y.id)

      #These just help make displaying and getting properties easier
      Base.show(io::IO, x::AbstractNode) = print(io, "Node($(x.id))")
```

```

Base.getindex(x::Node, s::Union{Symbol,String}) = get(x._props, Symbol(s),
↳get(_default, Symbol(s), missing))

#Setup edges. Work similar to nodes. These are directed edges (or arcs)
abstract type AbstractEdge end

#Edges are directed edges (or arcs) that are uniquely represent by their to/
↳from nodes and properties
struct Edge <: AbstractEdge
    from::Node
    to::Node
    _props::Dict{Symbol,<:Number}

    Edge(from::Node, to::Node) = new(from, to, Dict{Symbol,Number}())
    Edge(from::Node, to::Node, props::Dict{Symbol,<:Number}) = new(from, to,
↳props)
    Edge(from::Node, to::Node, ; kwargs...) = new(from, to, Dict(kwargs))
end

#This allows us to reference an Edge without necessarily needing to have access
↳to the edge object
#So, for example, they can be used as keys on the JuMP model
struct EdgeKey <: AbstractEdge
    from::AbstractNode
    to::AbstractNode
end

#This just enforces that nodes identifies the edge
Base.hash(x::AbstractEdge, y::UInt64) = hash([x.from, x.to], y)
Base.==(x::AbstractEdge, y::AbstractEdge) = x.from == y.from && x.to == y.to
Base.isequal(x::AbstractEdge, y::AbstractEdge) = isequal(x.from, y.from) &&
↳isequal(x.to, y.to)

#These just help make displaying Edges and getting properties easier
Base.show(io::IO, x::AbstractEdge) = print(io, "$(typeof(x))($ (x.from.id)->$(x.
↳to.id))")
Base.getindex(x::Edge, s::Union{Symbol,String}) = get(x._props, Symbol(s),
↳get(_default, Symbol(s), missing))

#Sets up a graph with a sparse structure.
#Also contains helper dicts so we can get arcs that go from or to a particular
↳node easily
struct SparseGraph
    nodes::Vector{Node}

```

```

edges::Vector{Edge}
fromEdges::Dict{Node,Set{Edge}}
toEdges::Dict{Node,Set{Edge}}

function SparseGraph(
    nodes::Vector{Node},
    edges::Vector{Edge}
)
    fromEdges = Dict{Node,Set{Edge}}{n => Set{Edge}(e for e in edges if n == e.from) for n in nodes}
    toEdges = Dict{Node,Set{Edge}}{n => Set{Edge}(e for e in edges if n == e.to) for n in nodes}
    new(nodes, edges, fromEdges, toEdges)
end
end

```

Graph Creation

We choose a grid pattern. Size and properties (usually cost) can be set as inputs.

```

[2]: using Random

function createGrid(nx::Int, ny::Int; propGenEdge=(to, from) -> Dict{Symbol,Number}())::SparseGraph
    #Create the nodes
    nodes = [Node(Symbol("$x,$y"), x, y) for x in 1:nx, y in 1:ny]
    edges = Edge[]

    #Generate edges going up/down/left/right
    for n in nodes
        for xd in (1, 0, -1)
            for yd in (1, 0, -1)
                newX = n.x + xd
                newY = n.y + yd
                if ((xd != 0 || yd != 0) && (xd * yd == 0))
                    && newX >= 1 && newX <= nx
                    && newY >= 1 && newY <= ny
                )
                to = nodes[Int(newX), Int(newY)]
                newEdge = Edge(n, to, propGenEdge(n, to))
                push!(edges, newEdge)
            end
        end
    end
    nodes = reduce(vcat, nodes) #flatten nodes
    return SparseGraph(nodes, edges)
end

```

```
;
```

This allows us to set costs however we want. We went with a uniform cost between 1 and 2.5 since speed limits are typically in the 25 to 65 mph range, which is approximates a $2.5\times$ gap.

```
[3]: function genGridCost(to, from)
      return Dict{:cost} => 2.5 * (rand() + 1))
end
;
```

This is our common function to plot diverse paths.

```
[4]: using CairoMakie

function plotPaths(ax::Axis, g::SparseGraph, paths::
    Vector{Vector{Edge}}=Vector{Edge}[])
    edgesInPath = length(paths) == 0 ? Set() : Set(reduce(vcat, paths))
    edgesNotInPath = [e for e in g.edges if !in(e, edgesInPath)]
    dispEdges = sort(g.edges, by=x -> in(x, edgesInPath) ? 1 : 0) #paint path
    edges last
    colors = [in(e, edgesInPath) ? :blue : :black for e in g.edges]

    linesegments!(ax,
        reduce(vcat, [[e.from.x, e.to.x] for e in g.edges]),
        reduce(vcat, [[e.from.y, e.to.y] for e in g.edges]),
        color=:black, linewidth=(reduce(vcat, [[e[:cost], e[:cost]] for e in g.
    edges]) .- 0.5) .^ 2)

    if (length(g.edges) < 1000)
        scatter!(ax, [n.x for n in g.nodes], [n.y for n in g.nodes],
    markersize=22)
    end

    for p in 1:length(paths)
        pathEdges = paths[p]
        arrows2d!(ax,
            [e.from.x for e in pathEdges], [e.from.y for e in pathEdges],
            [e.to.x - e.from.x for e in pathEdges], [e.to.y - e.from.y for e in
    pathEdges],
            #color=p, colormap=:glasbey_bw_minc_20_minl_30_n256, colorrange=(1,
    256),

            color=fill(p, length(pathEdges)),
            colormap=:glasbey_bw_minc_20_minl_30_n256,
            colorrange=(1, max(2, length(paths))),

```

```

        align=-0.25, lengthscale=0.65,
        shaftwidth=3,
        label="Path $p"
    )
end

return f
end
;

```

Model

Shortest Path Model

```

[5]: using JuMP, HiGHS
function getShortestPath(g::SparseGraph, from::AbstractNode, to::AbstractNode;
    ↪silent=true)

    function supply(n::AbstractNode)
        if (n == from)
            return 1 #Is the source node
        elseif (n == to)
            return -1 #is the sink node
        else
            return 0 #Is an intermediate node
        end
    end

    cost = Dict{e => e[:cost] for e in g.edges}

    m = Model(HiGHS.Optimizer)

    @variable(m, flow[g.edges] >= 0)

    @objective(m, Min, sum(cost[e] * flow[e] for e in g.edges))

    @constraint(m, flowConstr[n in g.nodes], sum(flow[e] for e in g.
    ↪fromEdges[n]) == sum(flow[e] for e in g.toEdges[n]) + supply(n))

    if (silent)
        set_silent(m)
    end

    optimize!(m)

    return [e for e in g.edges if value(flow[e]) > 0.00001]
end

```

```
;
```

Diversity Cost Model This gets the number of hops between edges in the graph, given a set of edges/arcs that are part of paths that already exist.

```
[6]: using JuMP, HiGHS

function getMinDistanceFromUsed(g::SparseGraph, used::Set{Edge}; silent=true)::
    Dict{Edge,Int}
    m = Model(HiGHS.Optimizer)

    @variable(m, dist[g.edges] >= 0)

    @objective(m, Max, sum([dist[e] for e in g.edges]))

    notUsed = [e for e in g.edges if !(e in used)]

    #Used edges start with distance 0 since they are on the path
    @constraint(m, usedZero[e in used], dist[e] == 0)
    #Unused edges use the minimum number of hops
    @constraint(m, unusedDist[to in notUsed, from in g.toEdges[to.from]],
        ↪dist[to] <= dist[from] + 1)

    if (silent)
        set_silent(m)
    end

    optimize!(m)

    if (is_solved_and_feasible(m))
        return Dict(e => value(dist[e]) for e in g.edges)
    else
        return nothing
    end
end
;
```

Next Diverse Path Model

```
[7]: function getDiverseShortPathTradeoff(g::SparseGraph, from::AbstractNode, to::
    AbstractNode, used::Set{Edge};
    diversityCost::Union{Function,Nothing}=(g, used) -> Dict(e => ((e in used) ?
    ↪ 1 : 0) for e in g.edges), #by default use edge overlap
    tradeOffWeight=1::Float64,
    silent=true)::Vector{Edge}

    function supply(n::AbstractNode)
        if (n == from)
```

```

        return 1 #Is the source node
    elseif (n == to)
        return -1 #is the sink node
    else
        return 0 #Is an intermediate node
    end
end

cost = Dict{e => e[:cost] for e in g.edges}
dCost = diversityCost(g, used)

m = Model(HiGHS.Optimizer)

@variable(m, flow[g.edges] >= 0)

#Essentially the same as a shortest path model except we add a tradeoff to
→make diverse edges
@objective(m, Min, sum([cost[e] * flow[e] for e in g.edges]) +
→tradeOffWeight * sum([dCost[e] * flow[e] for e in g.edges]))

@constraint(m, flowConstr[n in g.nodes], sum(flow[e] for e in g.
→fromEdges[n]) == sum(flow[e] for e in g.toEdges[n]) + supply(n))

if (silent)
    set_silent(m)
end

optimize!(m)

if (is_solved_and_feasible(m))
    return [e for e in g.edges if value(flow[e]) > 0.00001]
else
    return []
end
end
end
;

```

Final Iterative Model Here we define our diversity cost function, and our iterative procedure for finding diverse paths and solve it.

```

[8]: #Cost is based on the number of hopes
function myDiversityCost(g::SparseGraph, used::Set{Edge})
    dist = getMinDistanceFromUsed(g, used)
    return Dict{e => 1 / sqrt(dist[e] + 1) for e in keys(dist)}
end
;

```

```

[9]: function getDiversePaths(g::SparseGraph, from::AbstractNode, to::AbstractNode,
    ↪maxPaths=3::Number;
    diversityCost::Union{Function,Nothing}=nothing,
    tradeOffWeight=1::Float64,
    silent=true)::Vector{Vector{Edge}}

    paths = Vector{Edge}[]
    usedEdges = Set{Edge}()

    if (!silent)
        println("Solving Initial Shortest Path...")
    end

    path = getShortestPath(g, from, to, silent=silent)
    push!(paths, path)
    union!(usedEdges, path)

    for k in 1:maxPaths-1

        if (!silent)
            println("Solving Diverse path $k...")
        end

        path = getDiverseShortPathTradeoff(g, from, to, usedEdges,
    ↪diversityCost=diversityCost, tradeOffWeight=tradeOffWeight, silent=silent)

        if (length(path) == 0)
            if (!silent)
                println(stderr, "Only $(k) diverse path acheived")
            end
            break
        end

        push!(paths, path)
        union!(usedEdges, path)
    end
    return paths
end
;

```

This works fairly fast, so we can generate a 100×100 grid on various tradeoff weights and 7 total paths.

```

[10]: Random.seed!(4224)
baselineGrid = createGrid(100, 100, propGenEdge=genGridCost)
baselineWeights = [0, 0.1, 0.5, 2, 8, 16, 256]
baseline_path_num = 7

```

```

baselinePaths = [getDiversePaths(baselineGrid, NodeKey("2,2"),
    ↪NodeKey("97,97"), baseline_path_num, diversityCost=myDiversityCost,
    ↪tradeOffWeight=k) for k in baselineWeights]
;

```

Evaluation Functions These functions will help us evaluate our models in the discussion

```

[11]: using Statistics

#Calculate simple edge reuse
function pathOverlap(paths::Vector{Vector{Edge}})
    numOverlap = 0
    for k in 1:length(paths)
        for i in k+1:length(paths)
            for e1 in paths[k]
                for e2 in paths[i]
                    if (e1 == e2)
                        numOverlap += 1
                    end
                end
            end
        end
    end
    return numOverlap
end

#Get the average cost of all of the generated paths
getAvgPathCost(paths::Vector{Vector{Edge}}) = mean([sum([e[:cost] for e in p])
    ↪for p in paths])

#Get the difference in cost between the maximum cost path and the minimum cost
    ↪(shortest) path
getMaxPathDifference(paths::Vector{Vector{Edge}}) = maximum([sum([e[:cost] for
    ↪e in p]) for p in paths]) - minimum([sum([e[:cost] for e in p]) for p in
    ↪paths])

#Average diversity cost for each path
function getAvgHopsFromOtherPaths(g::SparseGraph, paths::Vector{Vector{Edge}})
    totalDistance = 0
    n = 0

    for k in 1:length(paths)
        thisPath = Set(e for e in paths[k])
        distance = getMinDistanceFromUsed(g, thisPath)
    end
end

```

```

        for i in (k+1):length(paths)
            n += length(paths[i])
            totalDistance += sum(distance[e] for e in paths[i])
        end
    end
    return totalDistance / n
end

#Generate the above statistics using the first n paths
function summarize(g::SparseGraph, solns::Vector{Vector{Vector{Edge}}}, npaths::
    Integer)
    return Dict(
        :num_paths => [length(paths[1:npaths]) for paths in solns],
        :avg_path_cost => [getAvgPathCost(paths[1:npaths]) for paths in solns],
        :overlap => [pathOverlap(paths[1:npaths]) for paths in solns],
        :cost_difference => [getMaxPathDifference(paths[1:npaths]) for paths in
    solns],
        :avg_hops => [getAvgHopsFromOtherPaths(g, paths[1:npaths]) for paths in
    solns],
    )
end
;

```

4. Discussion of Results

Our model is inherently a tradeoff between finding short paths and finding diverse paths, so we will present them as a set of pareto-optimal solutions given a set of weights.

We provide two diversity statistics and two shortest-path statistic.

Our most important shortest path statistic is the average length of each path. If we have shorter paths over all diverse paths, this statistic will be lower. We also find the difference between the longest and shortest paths to get an idea of how long the worst path is compared to the shortest to motivate us trying L_∞ regularization later.

We also compute the number of overlapping edges among the paths. We use this statistic because preventing edge overlap is a common way to create diverse paths, and we can show that for our grid structure that is not always sufficient. Lastly, we calculate the average number of hops we'd need to go from any edge from any path to any other edge in a different path, the lower this is the more diverse our paths are.

```

[12]: #Compute summaries for a number of dissimilar paths. Since our algorithm is
    iterative, we can calculate them based on our top k paths.

n_baseline_summary_paths = [2, 3, 5, 7]

baselineSummary = [summarize(baselineGrid, baselinePaths, k) for k in
    n_baseline_summary_paths]

```

```
;
```

As we'd expect, we can get more diverse paths at the expense of path cost. We are initially able to greatly increase our dissimilarity without adding too much cost to our paths.

We note that when generating 7 paths, there is a hitch we obtain shorter path costs and more dissimilarity at one point. This is likely an artifact of our incremental model design. New paths are only created based on existing ones, so different sets of prior dissimilar paths could allow better or worse paths in the future. As we increase the number of paths, the more likely this is.

We note that even though the other graphs depict tradeoffs, this Cost/Hops tradeoff graph is the only one we know is pareto-optimal because it is directly measuring the tradeoffs of our objective function.

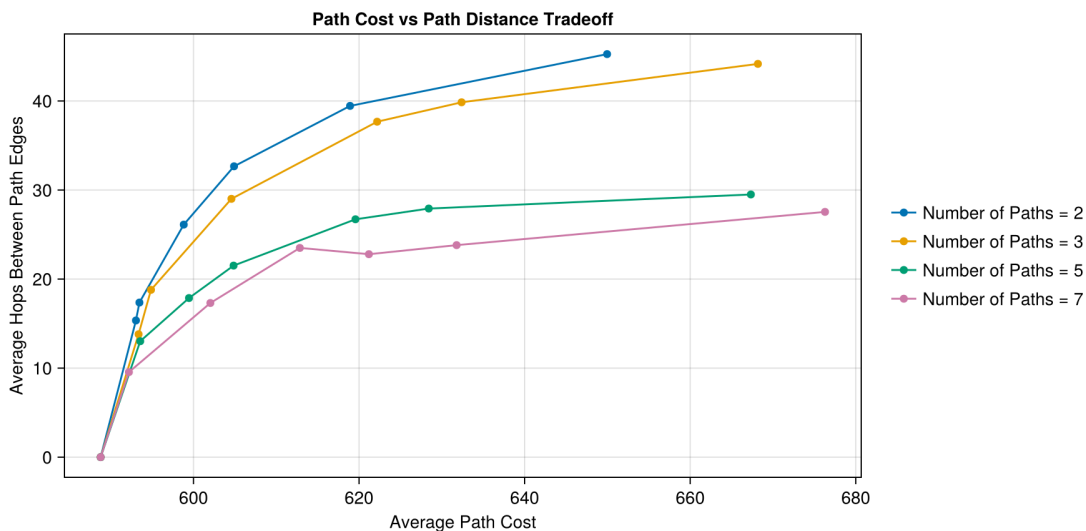
```
[13]: f = Figure(size=(900, 450))

ax2 = Axis(f[1, 1], xlabel="Average Path Cost", ylabel="Average Hops Between_
↳Path Edges",
          title="Path Cost vs Path Distance Tradeoff")

for (index, stats) in enumerate(baselineSummary)
    numPaths = n_baseline_summary_paths[index]
    scatterlines!(ax2, stats[:avg_path_cost], stats[:avg_hops], label="Number_
↳of Paths = $numPaths")
end

Legend(f[1, 2], ax2, framevisible=false)
f
```

[13]:



Edge overlap decreases extremely rapidly as we add weight to finding diverse paths. For a large

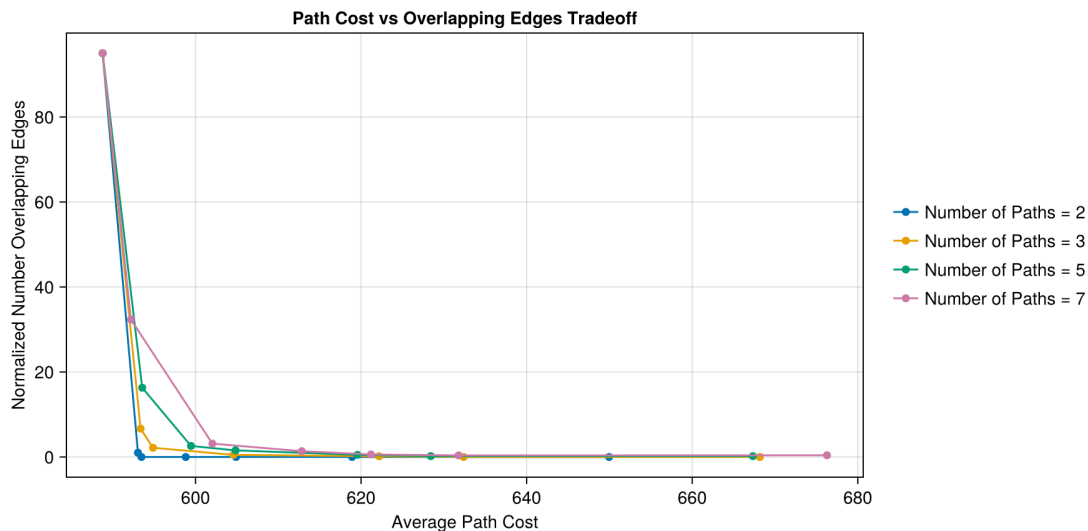
number of paths, overlap cannot be completely removed because there are only so many edges in and out of the source and sink nodes. We note our dissimilarity measure can still provide large changes even when overlap is near zero, showing that it is more useful over a wider range of diverse paths.

```
[14]: f = Figure(size=(900, 450))
ax1 = Axis(f[1, 1], xlabel="Average Path Cost", ylabel="Normalized Number of
↳Overlapping Edges",
title="Path Cost vs Overlapping Edges Tradeoff")

for (index, stats) in enumerate(baselineSummary)
    numPaths = n_baseline_summary_paths[index]
    scatterlines!(ax1, stats[:avg_path_cost], stats[:overlap] ./ (numPaths *
↳(numPaths - 1)), label="Number of Paths = $numPaths")
end

Legend(f[1, 2], ax1, framevisible=false)
f
```

[14]:



A similar story occurs when looking at the cost of the longest path minus the cost of the shortest path. Since we are not directly optimizing this measure, the graphs are less smooth. There seems to be a level of dissimilarity where a significant increase in the path costs shows up. However, when it comes to overlap there's not much additional story to see.

```
[15]: f = Figure(size=(900, 450))
ax4 = Axis(f[1, 1], xlabel="Maximum Minus Minimum Path Length", ylabel="Average
↳Hops Between Path Edges",
title="Path Difference vs Path Distance Tradeoff")
```

```

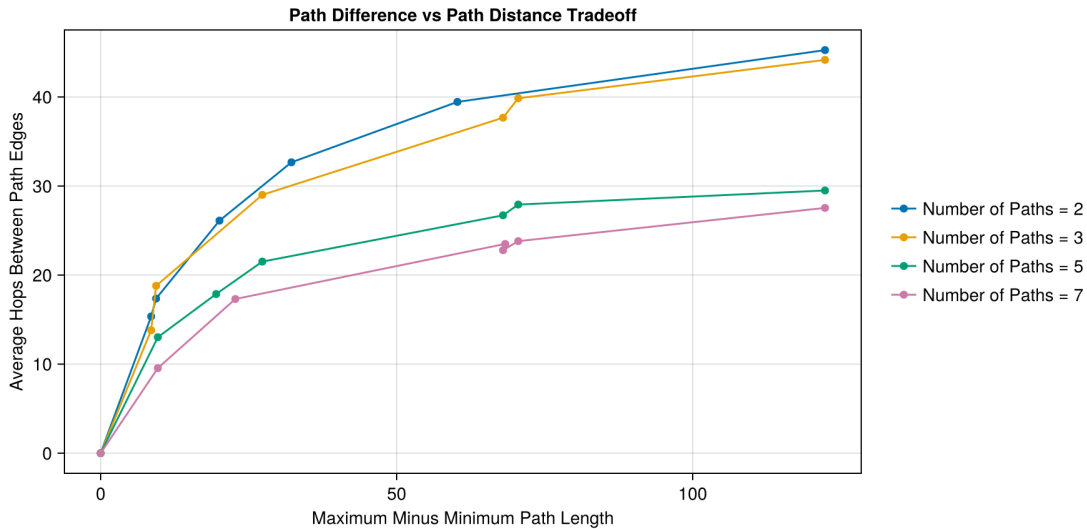
for (index, stats) in enumerate(baselineSummary)
    numPaths = n_baseline_summary_paths[index]
    scatterlines!(ax4, stats[:cost_difference], stats[:avg_hops], label="Number_
    of Paths = $numPaths")
end

Legend(f[1, 2], ax4, framevisible=false)

f

```

[15]:



```

[16]: f = Figure(size=(900, 450))

ax3 = Axis(f[1, 1], xlabel="Maximum Minus Minimum Path Length",
    ylabel="Normalized Number of Overlapping Edges Plus One",
    title="Path Difference vs Overlapping Edges Tradeoff")

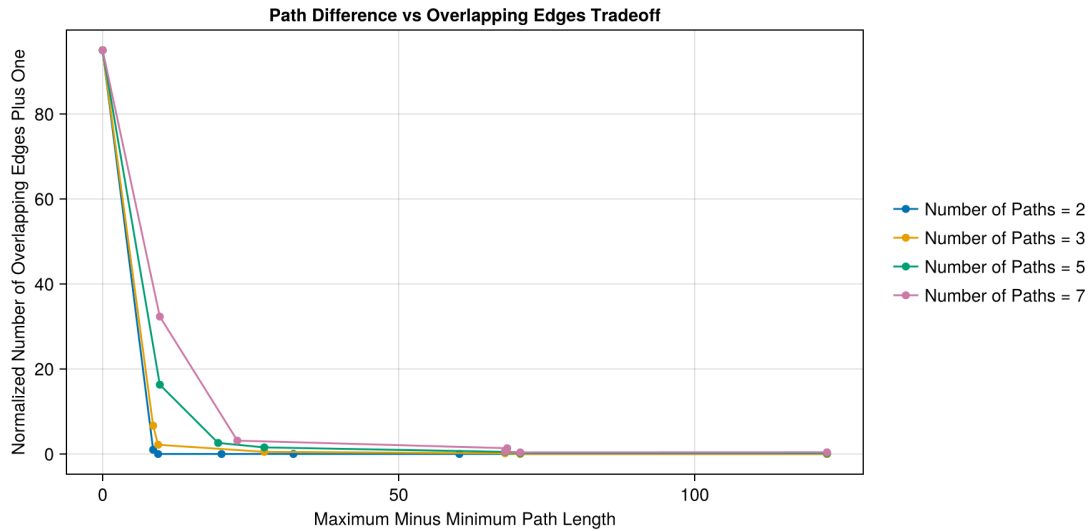
for (index, stats) in enumerate(baselineSummary)
    numPaths = n_baseline_summary_paths[index]
    scatterlines!(ax3, stats[:cost_difference], stats[:overlap] ./ (numPaths *
    (numPaths - 1)), label="Number of Paths = $numPaths")
end

Legend(f[1, 2], ax3, framevisible=false)

f

```

[16]:



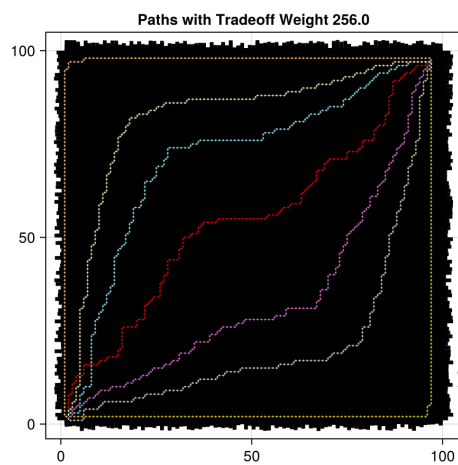
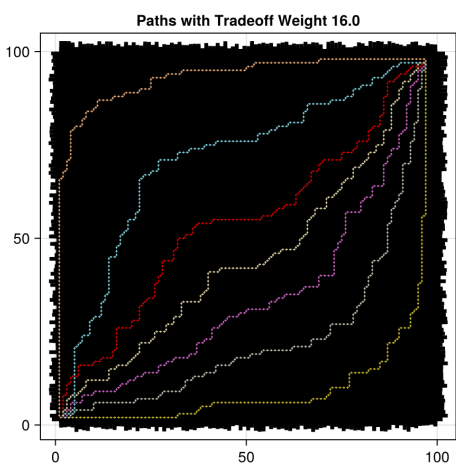
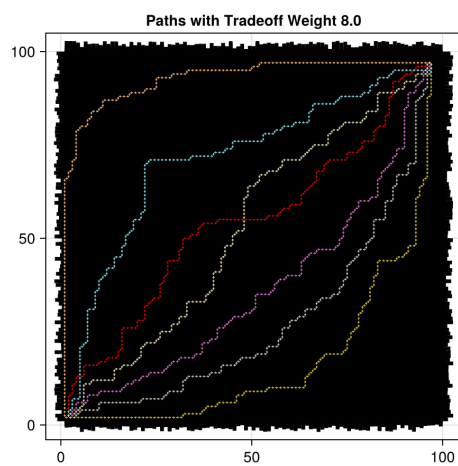
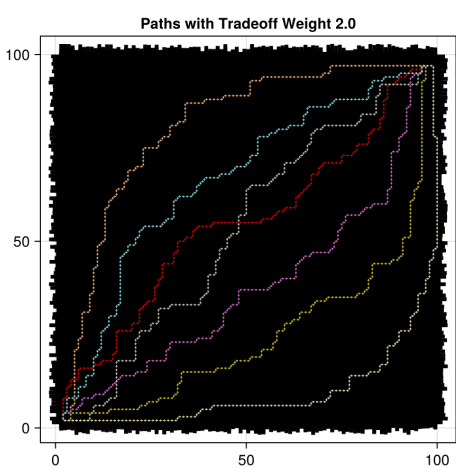
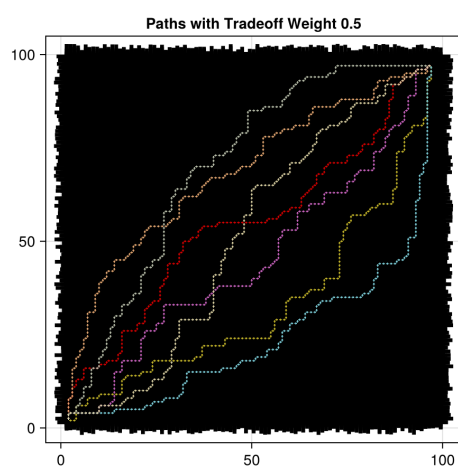
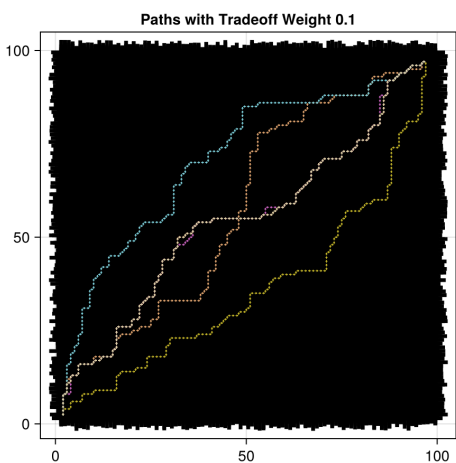
We can also visualize our paths as we increase the weight we put to diversity. When we care nothing for diversity, all paths take the shortest path. As we increase the weight there is less crossing of the paths and they are spread farther apart until they are essentially as far apart as possible.

```
[17]: f = Figure(size=(900, 1800))

for k in 1:length(baselinePaths)
    adjK = k - 1
    tradeoffWeight = baselineWeights[k]
    lastAx = Axis(f[adjK÷2+1, adjK%2+1], title="Paths with Tradeoff Weight_
↪$tradeoffWeight")
    plotPaths(lastAx, baselineGrid, baselinePaths[k])
end

f
```

[17]:



There’s only so many ways to leave the source node in the graph, so there is some crossing even with a very high weight. Some of this could theoretically be avoided by calculating the solution non-iteratively, but as we will see in the excursions a naive MIP program is too slow.

Even though the model works well, we are not strictly speaking getting the k most diverse short paths possible. Due to the iterative nature of our model, we are only getting the most diverse path given the other paths. It could be that making different choices could lead to overall better outcomes. In addition, we aren’t able to use regularization with this model to try and make the path lengths similar since we generate each path one at a time.

5. Excursions

Excursion 1: Hilliness

We consider the hilliness or grade of each arc by letting each arc have a steepness draw from a Normal distribution. This excursion is motivated by trying to approximate fuel efficiency while traveling in a vehicle. Vehicles are much less fuel efficient when traveling on steep roads versus flat or downhill roads. Therefore, this excursion can be thought of as, “find N dissimilar short paths when considering how roads hilliness affects fuel efficiency.”

Excursion 1 Explained For each arc $a \in A$, let the g_a be Normally distributed with mean zero and standard deviation 0.10. Let $\bar{g} = 0.05$. Note: we initially began with a standard deviation of 0.05, but this did not produce interesting results. There was not enough arcs with road grades greater than 0.05 to have the constraint matter.

Define

$$\mathbf{1}_{a,\bar{g}} = \begin{cases} 1 & \text{if } g_a > \bar{g} \\ 0 & \text{otherwise,} \end{cases}$$

where $\mathbf{1}_{a,\bar{g}}$ counts if an arc $a \in A$ has grade greater than 5%. Let $M_{\bar{g}}$ be the maximum number of above-threshold arcs allowed in a shortest path. The model formulation of Excursion 1 is the same as the baseline model, except for two differences. First, because we are adding a constraint the model may not be totally unimodular anymore, so we utilize binary decision variables. Second, now that we are dealing with integer decision variables, we have an initial shortest path integer program (ISP IP) and a diverse shortest path integer program (DSP IP). Third, for the ISP IP and at iteration $k \in \{2, \dots, N\}$ of the DSP IP, we add the constraint

$$\sum_{a \in A} \mathbf{1}_{a,\bar{g}} x_a \leq M_{\bar{g}}.$$

Experimental Design Elements We now outline the experimental design used to analyze the Excursion 1 model.

1. Fix a grid size and source and sink nodes.
2. Fix a seed for reproducibility.
3. Vary λ .
4. Vary $M_{\bar{g}}$.
5. Vary number of diversified shortest paths N .

```

[18]: using Random
using Statistics
using JuMP
using HiGHS
using CairoMakie

# Grade add-on for baseline graphs

const GRADE_THRESHOLD = 0.05
const GRADE_SD = 0.1

function addGrades!(
    g::SparseGraph;
    grade_sd::Float64=GRADE_SD,
    grade_threshold::Float64=GRADE_THRESHOLD
)
    for e in g.edges
        grade = grade_sd * randn()
        e._props[:grade] = grade
        e._props[:steep] = grade > grade_threshold ? 1.0 : 0.0
    end

    return g
end

function steepIndicator(e::Edge)::Int
    return Int(round(e[:steep]))
end

function pathSteepCount(path::Vector{Edge})
    return sum(steepIndicator(e) for e in path)
end
;

```

The next code block alters the initial shortest path LP into the ISP IP with a grade constraint.

```

[19]: # Initial shortest path IP with grade constraint

function getShortestPathGradeConstrained(
    g::SparseGraph,
    from::AbstractNode,
    to::AbstractNode;
    M_grade::Int,
    silent=true
)::Vector{Edge}

    function supply(n::AbstractNode)
        if n == from

```

```

        return 1
    elseif n == to
        return -1
    else
        return 0
    end
end

cost = Dict{e => e[:cost] for e in g.edges}
steep = Dict{e => steepIndicator(e) for e in g.edges}

m = Model(HiGHS.Optimizer)
set_silent(m)
# Binary variables are used because the extra grade-budget constraint
# turns the problem into a resource-constrained shortest path problem.
@variable(m, x[g.edges], Bin)

@objective(m, Min, sum(cost[e] * x[e] for e in g.edges))

@constraint(m, flowConstr[n in g.nodes], sum(x[e] for e in g.fromEdges[n]) -
↪ == sum(x[e] for e in g.toEdges[n]) + supply(n))
@constraint(m, gradeBudget, sum(steep[e] * x[e] for e in g.edges) <=
↪ M_grade)

optimize!(m)

if is_solved_and_feasible(m)
    return [e for e in g.edges if value(x[e]) > 0.5]
else
    return Edge[]
end
end
;

```

The next function will be employed during the dissimilar shortest paths loop of Excursion 1. It is the diverse shortest path integer programming model after the initial shortest path problem integer program is solved for Excursion 1.

```

[20]: # Diverse shortest path IP with grade constraint

function getDiverseShortPathTradeoffGradeConstrained(
    g::SparseGraph,
    from::AbstractNode,
    to::AbstractNode,
    used::Set{Edge};
    diversityCost::Union{Function,Nothing}=nothing,
    tradeOffWeight::Float64=1.0,

```

```

M_grade::Int,
silent=true
)::Vector{Edge}

function supply(n::AbstractNode)
    if n == from
        return 1
    elseif n == to
        return -1
    else
        return 0
    end
end

cost = Dict{e => e[:cost] for e in g.edges}
steep = Dict{e => steepIndicator(e) for e in g.edges}

if diversityCost === nothing
    dCost = Dict{e => ((e in used) ? 1.0 : 0.0) for e in g.edges}
else
    dCost = diversityCost(g, used)
end

m = Model(HiGHS.Optimizer)

@variable(m, x[g.edges], Bin)

@objective(m, Min, sum(cost[e] * x[e] for e in g.edges) + tradeOffWeight *
↳sum(dCost[e] * x[e] for e in g.edges))

@constraint(m, flowConstr[n in g.nodes], sum(x[e] for e in g.fromEdges[n])
↳== sum(x[e] for e in g.toEdges[n]) + supply(n))

@constraint(m, gradeBudget, sum(steep[e] * x[e] for e in g.edges) <=
↳M_grade)

if silent
    set_silent(m)
end

optimize!(m)

if is_solved_and_feasible(m)
    return [e for e in g.edges if value(x[e]) > 0.5]
else
    return Edge[]
end

```

```
end
;
```

The next function actually obtains the dissimilar shortest paths for Excursion 1.

```
[21]: # Sequential diverse paths for Excursion

function getDiversePathsGradeConstrained(
    g::SparseGraph,
    from::AbstractNode,
    to::AbstractNode,
    maxPaths::Int=3;
    diversityCost::Union{Function,Nothing}=nothing,
    tradeOffWeight::Float64=1.0,
    M_grade::Int,
    silent=true)::Vector{Vector{Edge}}

    paths = Vector{Edge}[]
    usedEdges = Set{Edge}()

    path = getShortestPathGradeConstrained(g, from, to; M_grade=M_grade,
    ↪silent=silent)

    if length(path) == 0
        println(stderr, "No feasible initial path achieved for M_grade =
    ↪$M_grade.")
        return paths
    end

    push!(paths, path)
    union!(usedEdges, path)

    for k in 1:(maxPaths-1)

        path = getDiverseShortPathTradeoffGradeConstrained(g, from, to,
    ↪usedEdges; diversityCost=diversityCost,
            tradeOffWeight=tradeOffWeight, M_grade=M_grade, silent=silent)

        if length(path) == 0
            println(stderr, "Only $(k) diverse paths achieved for M_grade =
    ↪$M_grade.")
            break
        end

        push!(paths, path)
        union!(usedEdges, path)
    end
end
```

```

    return paths
end
;

```

The next code block consists of some helper functions to generate metrics and graphs for the Excursion 1 results.

```

[22]: function safeAvgPathCost(paths::Vector{Vector{Edge}})
    if length(paths) == 0
        return NaN
    end

    return getAvgPathCost(paths)
end

function avgSteepArcs(paths::Vector{Vector{Edge}})
    if length(paths) == 0
        return NaN
    end

    return mean([pathSteepCount(p) for p in paths])
end

function uniqueEdgeCount(paths::Vector{Vector{Edge}})
    if length(paths) == 0
        return 0
    end

    usedEdges = Set{Edge}()

    for p in paths
        union!(usedEdges, p)
    end

    return length(usedEdges)
end

function summarizeExcursion(g::SparseGraph, paths::Vector{Vector{Edge}};
    ↪target_paths::Int)
    return Dict(
        :num_paths => length(paths),
        :found_all_paths => length(paths) == target_paths ? 1.0 : 0.0,
        :avg_path_cost => safeAvgPathCost(paths),
        :avg_steep_arcs => avgSteepArcs(paths),
        :unique_edges_used => uniqueEdgeCount(paths))
end

```

```

function thirdPathObjective(g::SparseGraph, paths::Vector{Vector{Edge}};
    diversityCost=myDiversityCost, lambda::Float64=1.0)

    third_path = paths[3]

    usedEdges = Set{Edge}()
    union!(usedEdges, paths[1])
    union!(usedEdges, paths[2])

    dCost = diversityCost(g, usedEdges)
    path_cost = sum(e[:cost] for e in third_path)
    diversity_penalty = sum(dCost[e] for e in third_path)
    objective_value = path_cost + lambda * diversity_penalty

    return Dict(
        :path_cost => path_cost,
        :diversity_penalty => diversity_penalty,
        :objective_value => objective_value)
end
;

```

In the next code block we execute the previously mentioned experimental design to analyze the Excursion 1 model. Three choices of tradeoff weights λ are chosen, as well as three choices for the grade constraint M and the amount of dissimilar paths to solve for. A graph size of 75×75 is chosen for greater computational efficiency while still allowing for interesting results.

```

[ ]: Random.seed!(1)

# Generate slightly smaller grid for efficiency
myGrid = createGrid(75, 75, propGenEdge=genGridCost)

# Add grades as an add-on after the baseline graph is created
addGrades!(myGrid; grade_sd=GRADE_SD, grade_threshold=GRADE_THRESHOLD)

source = NodeKey("2,2")
sink = NodeKey("72,72")

target_paths = [3,5,7]

weights = [0.1, 1.0, 2.5]

M_values = [5, 20, 35]

excursionResults = Dict()

for N in target_paths

```

```

    for M in M_values
        for lambda in weights
            println()
            #println("Running Excursion 1: Number of paths = $N, M_grade = $M, 
↪ lambda = $lambda")
            paths = getDiversePathsGradeConstrained(myGrid,source,sink,N;
                ↪
↪ diversityCost=myDiversityCost,tradeOffWeight=lambda,M_grade=M,silent=true)
            metrics = summarizeExcursion(myGrid,paths;target_paths=N)
            excursionResults[(N, M, lambda)] = Dict(:paths => paths,:metrics =>↪
↪ metrics)
            end
        end
    end
end
;

```

We now analyze the results of the experimental design for Excursion 1 through graphs and figures.

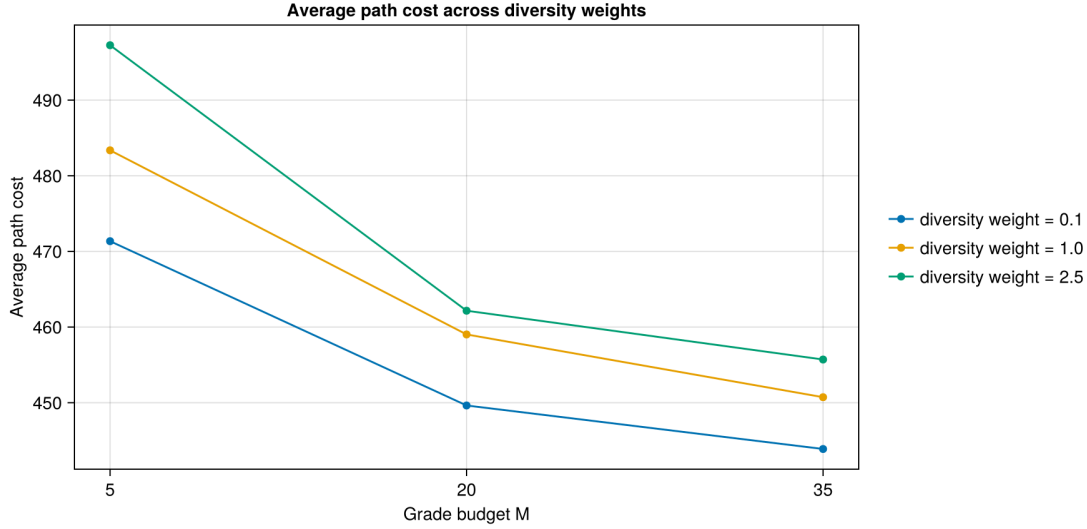
```

[24]: # Overlay average path cost for each lambda on one graph for N = 5
fig1 = Figure(size=(900, 450))
ax1 = Axis(fig1[1, 1],xlabel="Grade budget M",ylabel="Average path cost",
    title="Average path cost across diversity weights",xticks=M_values)

for lambda in weights
    avg_cost_by_M = [excursionResults[(5, M, lambda)][:metrics][:avg_path_cost]↪
↪ for M in M_values]
    scatterlines!(ax1, M_values, avg_cost_by_M, label="diversity weight =↪
↪ $lambda")
end

#axislegend(ax1, position=:rb)
Legend(fig1[1, 2], ax1, framevisible = false)
display(fig1);

```

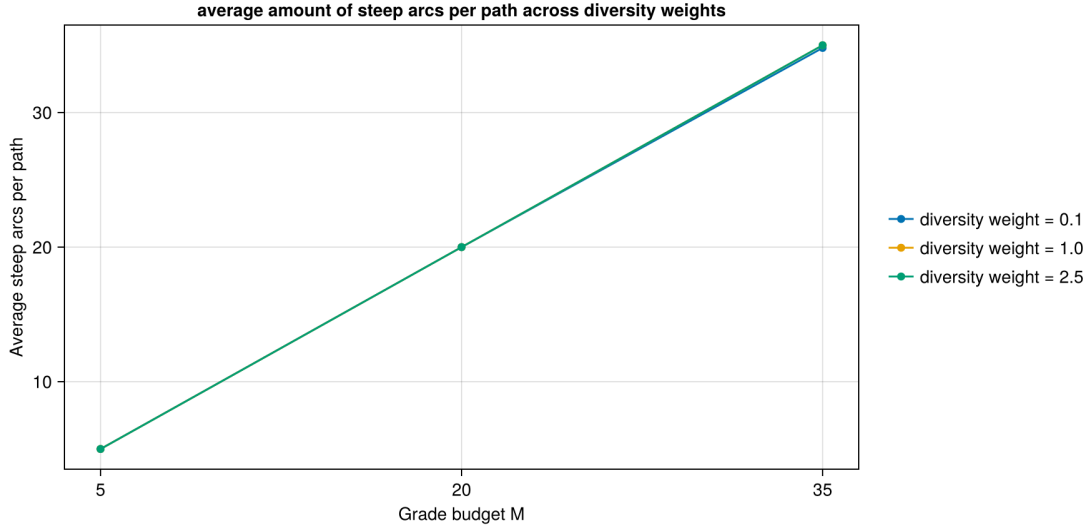


The previous figure shows the average path cost for $N = 5$ diverse paths across the diversity tradeoff weight $\lambda \in \{0.1, 1, 2.5\}$ for each allowable amount of arcs (i.e. grade budget M) with grade equal to or exceeding 5%. As expected, the more diversity required between the shortest path solutions, the greater the average path cost. Moreover, the graph demonstrates that as the grade budget M increases the average path cost decreases. This makes sense, because as M grows, the constraint is less tight, and so the model may use more steep arcs, allowing for the possibility of cheaper arcs to be chosen. In other words, the feasible set grows and the model finds cheaper solutions.

```
[25]: # Overlay average steep arcs per path for each lambda on one graph for N = 5
fig2 = Figure(size=(900, 450))
ax2 = Axis(fig2[1, 1], xlabel="Grade budget M", ylabel="Average steep arcs per_
    ↪path",
            title="average amount of steep arcs per path across diversity_
    ↪weights", xticks=M_values)

for lambda in weights
    avg_steep_by_M = [excursionResults[(5, M, lambda)][:metrics][:
    ↪avg_steep_arcs] for M in M_values]
    scatterlines!(ax2, M_values, avg_steep_by_M, label="diversity weight =_
    ↪$lambda")
end

Legend(fig2[1, 2], ax2, framevisible = false)
display(fig2);
```

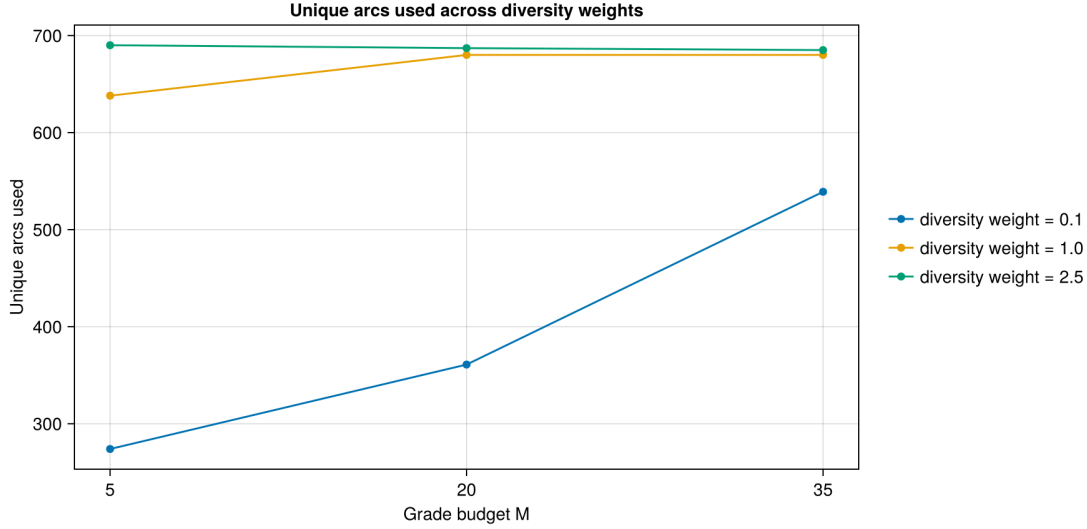


The previous figure shows the average steep arcs per path for $N = 5$ paths across the diversity tradeoff weight $\lambda \in \{0.1, 1, 2.5\}$ for each amount of allowed steep arcs M . Unsurprisingly, the figure shows that the grade budget constraint is tight, and the value of λ does not change the graph. Instead, all λ values produce essentially the same average steep arc count.

```
[26]: # Graph 3: Overlay unique arcs used for each lambda on one graph for N = 5
fig3 = Figure(size=(900, 450))
ax3 = Axis(fig3[1, 1], xlabel="Grade budget M", ylabel="Unique arcs used",
           title="Unique arcs used across diversity weights", xticks=M_values)

for lambda in weights
    unique_edges_by_M = [excursionResults[(5, M, lambda)][:metrics][:
↪ unique_edges_used] for M in M_values]
    scatterlines!(ax3, M_values, unique_edges_by_M, label="diversity weight =_
↪ $lambda")
end

Legend(fig3[1, 2], ax3, framevisible = false)
display(fig3);
```

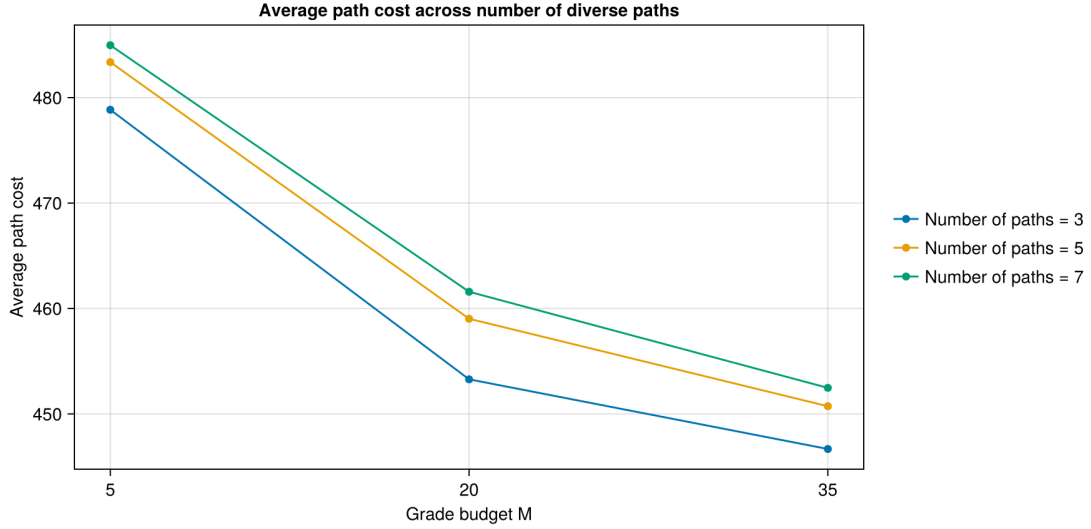


The previous figure shows the amount of unique arcs used. We see great similarity between $\lambda = 0.1$ and $\lambda = 1.0$. The case of a lesser diversity tradeoff weight of 0.1 is vastly different from the other two λ values. This shows the diversity penalty greatly affecting the objective function, because we see a higher λ increases the uniqueness of the solution. Predictably, for the $\lambda = 0.1$ and slightly for the $\lambda = 0.1$ case, as the grade budget M grows (i.e., as the feasible set grows), more unique edges are utilized because the model is less restricted.

```
[27]: # Overlay average path cost for each lambda on one graph for N = 5
fig4 = Figure(size=(900, 450))
ax4 = Axis(fig4[1, 1], xlabel="Grade budget M", ylabel="Average path cost",
           title="Average path cost across number of diverse_
           ↪paths", xticks=M_values)

for N in target_paths
    avg_cost_by_M = [excursionResults[(N, M, 1)][:metrics][:avg_path_cost] for_
    ↪M in M_values]
    scatterlines!(ax4, M_values, avg_cost_by_M, label="Number of paths = $N")
end

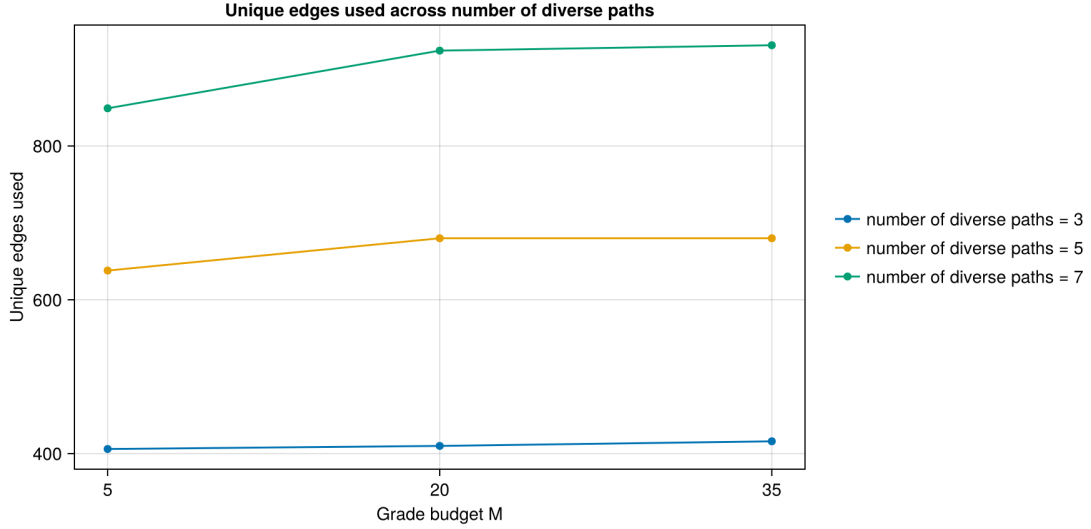
Legend(fig4[1, 2], ax4, framevisible = false)
display(fig4);
```



For $\lambda = 1$, this figure shows the average path cost for each grade budget M for each number of paths $N = 3, 5, 7$. As the amount of diverse paths increase the average path cost increases, because the diversity tradeoff in the objective function forces later paths found as N grows to become more expensive. As before, as the grade budget M increase the feasible set grows because more steep arcs are allowed.

```
[28]: fig = Figure(size = (900, 450))
ax = Axis(fig[1, 1], xlabel = "Grade budget M", ylabel = "Unique edges_
    ↪used", title = "Unique edges used across number of diverse paths",
        xticks = M_values)

for N in target_paths
    unique_edges_by_M = [excursionResults[(N, M, 1)][:metrics][:
    ↪unique_edges_used] for M in M_values]
    scatterlines!(ax, M_values, unique_edges_by_M, label = "number of diverse_
    ↪paths = $N")
end
Legend(fig[1, 2], ax, framevisible = false)
display(fig);
```



For $\lambda = 1$, the figure above shows the change in unique edges used based on the allowed amount of steep arcs M and the number of diverse paths being solved for by the solution procedure. Within each solution procedure, the number of unique arcs does not vary greatly, besides the case of $N = 7$. It makes sense that this case would have the greatest change from $N = 3$ to $N = 7$, because as the feasible set increases in size, the solution procedure needs to generate more diverse paths, which directly entails using more unique arcs. Interestingly, the unique arcs used varies vastly depending on the number of diverse paths being solved for. This is a result of solving for more paths. Because more paths are being solved for, the number of arcs utilized increases. Because of the diversity tradeoff present in the objective, this will force these arcs to be unique.

Below we plot the dissimilar shortest paths obtained for the case of tradeoff weight $\lambda = 1$ and $N = 3$ shortest paths for each of the M grade budget values 3, 20, and 35.

```
[48]: # Now directly plot the actual path sets for each case in that graph
N = 3
lambda = 1.0

for M in M_values
    paths = excursionResults[(N, M, lambda)][:paths]
    metrics = excursionResults[(N, M, lambda)][:metrics]
    println("N = $N, M = $M, diversity tradeoff weight = $lambda")
    fig = Figure(size=(900, 450))
    ax = Axis(fig[1, 1], title="N = $N, M = $M, diversity weight = $lambda")
    plotPaths(ax, myGrid, paths)
    display(fig)
end

paths_M5 = excursionResults[(N, 5, lambda)][:paths]
paths_M20 = excursionResults[(N, 20, lambda)][:paths]
```

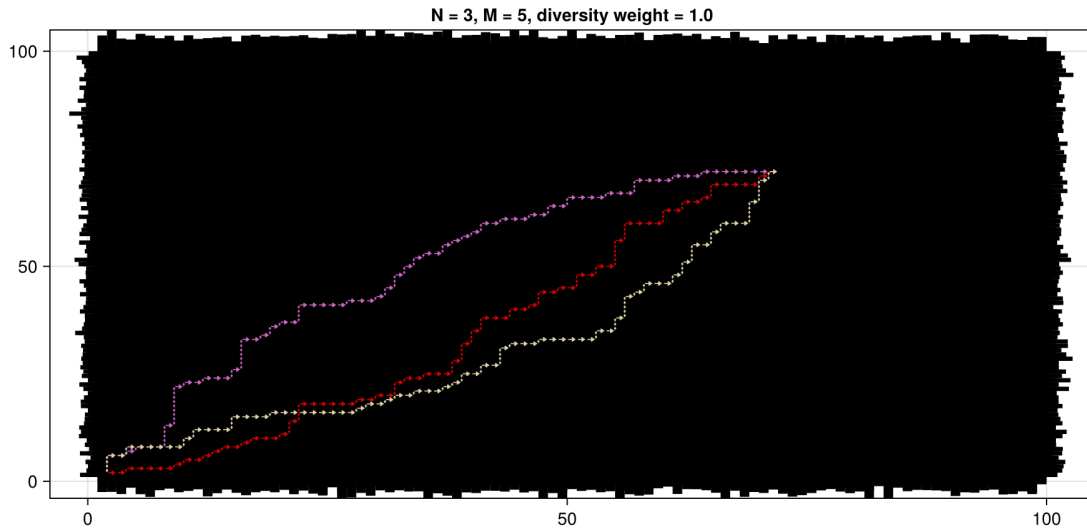
```

obj_M5 = thirdPathObjective(myGrid, paths_M5; lambda=lambda)
obj_M20 = thirdPathObjective(myGrid, paths_M20; lambda=lambda)

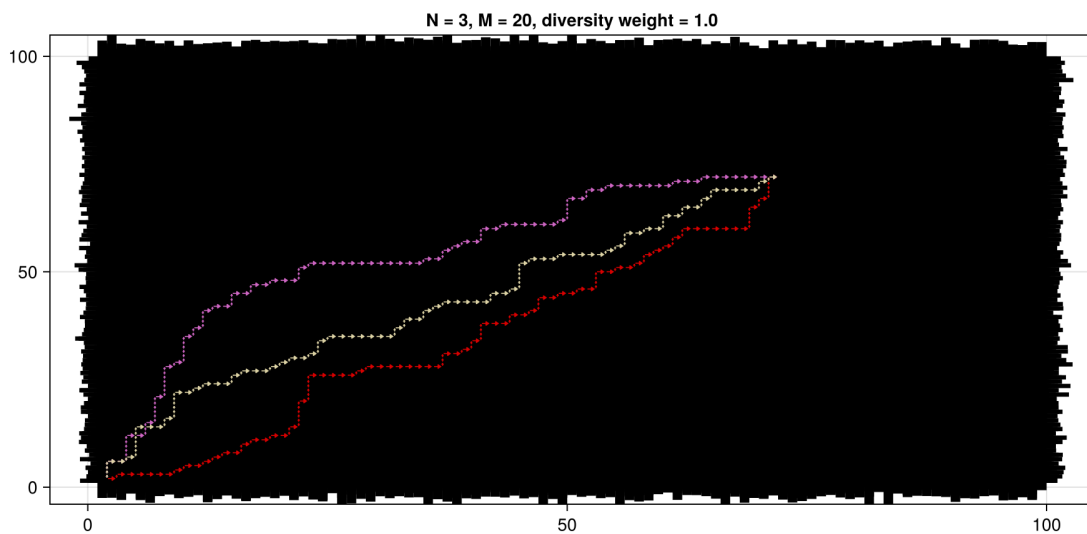
println("M = 5 third path objective: ", obj_M5[:objective_value])
println()
println("M = 20 third path objective: ", obj_M20[:objective_value])

```

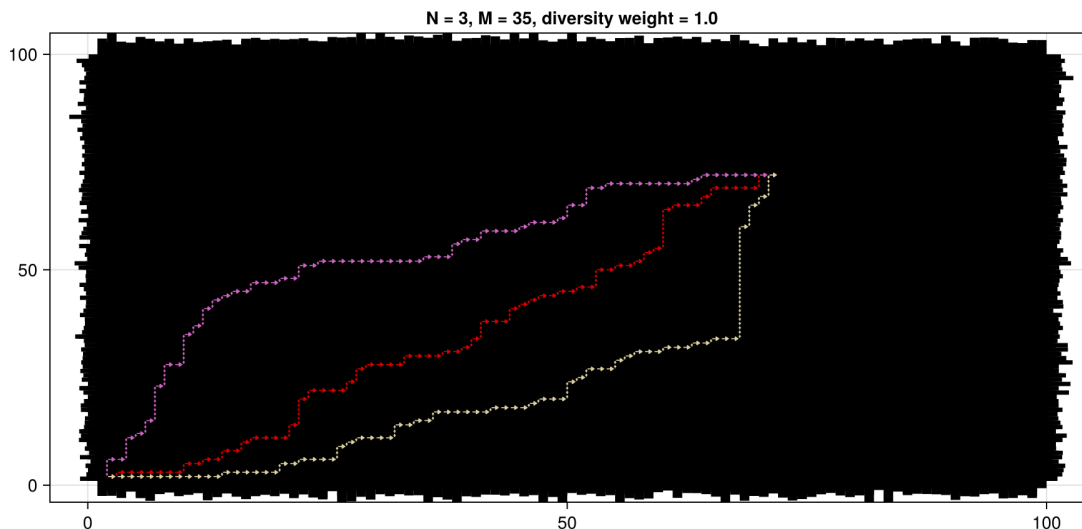
N = 3, M = 5, diversity tradeoff weight = 1.0



N = 3, M = 20, diversity tradeoff weight = 1.0



$N = 3$, $M = 35$, diversity tradeoff weight = 1.0



$M = 5$ third path objective: 548.5075191376685

$M = 20$ third path objective: 516.6503294053134

The previous three figures show the actual paths obtained by the model for $N = 3$ and $\lambda = 1$ and across the values of M of 5, 20, and 35. While the $M = 5$ and $M = 20$ graphs do not show much difference, the printed objective function values show that for the third path, because the $M = 20$ case is lower, the model is properly finding cheaper paths even without being greatly spread out. That is, even though visually we cannot tell much difference between the $M = 5$ and $M = 20$ cases, the more relaxed constraint is able to find a cheaper third path.

For $M = 35$, we see the paths being much more greatly spread out. This makes sense, because as the grade budget constraint becomes less binding, the model will be allowed to produce more visually distinct paths. That is, the tradeoff objective is less restricted when M is less restrictive.

Excursion 1 Summary In this excursion, we extended the baseline diverse shortest path model by adding a hilliness constraint. Each arc was assigned a random grade, and arcs above the grade threshold were counted as steep. We then limited the number of steep arcs allowed in each path using a grade budget M . Because this additional resource constraint changes the structure of the shortest path problem, we used binary decision variables and solved integer programs instead of the linear programs used in the baseline model.

The results show that the grade budget creates the expected tradeoff. When M is small, the model is forced to avoid steep arcs, which restricts the feasible set and increases average path cost. As (M) increases, the constraint becomes less binding, so the model can use more steep arcs and generally finds cheaper paths. The plots also show that increasing the diversity weight λ tends to

increase path cost and increase the number of unique arcs used, which is consistent with the goal of producing more dissimilar paths.

Overall, Excursion 1 shows that the baseline model can be adapted to include realistic route preferences such as avoiding steep roads. However, this comes at a computational cost because the grade-constrained version requires integer programming. It also preserves the same limitation as the baseline sequential method. That is, each new path is optimized relative to the paths already chosen, so the final set of paths may not be globally optimal as a collection.

Excursion 2: Simultaneous Model

We saw that the baseline model tends to work well and quickly, but the iterative method means that there could be sets of paths that are both shorter and more diverse. If there are multiple shortest paths, it could be that one shortest path allows for more diversity than another. It could also be that we could choose paths where none are the shortest path would give us better overall paths.

The iterative model also doesn't allow us to add additional regularization on the path lengths. While the short path objectives can be seen as L_1 regularization, it still isn't regularizing all of the paths at once. In addition, L_∞ regularization, which we could use to try and make all the paths close to the same cost, is similarly hamstrung. This is a bit different from typical regularization which would seek to minimize the number of edges chosen and not the costs, but it is still a form of regularization. We do this because trying to keep the total path costs more or less equal is something people might want in a real-world scenario because if one path was significantly longer a user would not be likely to select it.

These are weaknesses that can be addressed with a model that tries to find all of the diverse paths simultaneously in a single model instead of one at a time. However, this comes at an extreme cost of efficiency since we not only have to solve a mixed-integer program, but we also have to find the minimum number of hops between every pair of edges and add many constraints so that we only take into account the hop costs between edges that are part of different paths.

Simultaneous Model Formulation We also have a formulation that is not iterative and solves the diverse paths simultaneously in a single model. This requires a mixed-integer program since it is no longer totally-unimodular. However, we are able to add L_∞ regularization to the path lengths to try and keep them similar.

1. Flow Changes For this model, we are creating a k paths, so we change the x variables to be indexed by path and arc, $x_{n,a}$. These must also be binary variables.

2. Shortest Path Construction This model allows a bit more flexibility as to what the path lengths must be. We constrain the first diverse path to be within a factor p of the shortest path. We let the edges of the shortest path be y_a for all $a \in A$. In addition, we will put a large weight l as a factor of the shortest path objective cost to ensure that it is the highest priority path to be optimized. In practice, we set l to be the sum of all the edge costs and tradeoff weights to guarantee that the minimum objective must at least find that shortest path.

3. Diversity Cost Our costs are also indexed by the n -th path they are a part of, $d_{n,a}$ and are now variables instead of data because the model set all the paths. Cost must only exist between arcs

that belong to different paths. To denote this, we introduce variables $f_{n_1, a_1, n_2, a_2} \quad \forall 1 \leq n_1, n_2 \leq N, n_1 \neq n_2, a_1, a_2 \in A$.

As data, our computed cost between arcs of different paths is $o_{a_1, a_2} \quad \forall a_1, a_2 \in A$

4. Regularization The new model also supports L_∞ regularization to keep path length close together if possible. We use convexity to compute $t = \max(\sum_{a \in A} c_a x_{k,a} \quad \forall 1 \leq k \leq N)$ and let α be the weight of the regularization.

Baseline Simultaneous N-Diverse Shortest Paths Model

$$\min \quad l \sum_{a \in A} (c + a y_a) + \sum_{a \in A, 1 \leq n \leq N} c_a x_{n,a} + \lambda \sum_{a \in A, 1 \leq n \leq N} d_{n,a} x_{n,a} + \alpha t \quad (14)$$

$$\text{s.t.} \quad \sum_{a \in \delta^{\text{out}}(v)} y_a - \sum_{a \in \delta^{\text{in}}(v)} y_a = b_v \quad \text{for all } v \in V, \quad (15)$$

$$\sum_{a \in \delta^{\text{out}}(v)} x_{n,a} - \sum_{a \in \delta^{\text{in}}(v)} x_{n,a} = b_v \quad \text{for all } v \in V, 1 \leq n \leq N \quad (16)$$

$$p \sum_{a \in A} (c_a y_a) \leq \sum_{a \in A} c_a x_{1,a} \quad (17)$$

$$f_{n_1, a_1, n_2, a_2} \leq x_{n_1, a_1} \quad \text{for all } a_1, a_2 \in A, 1 \leq n_1, n_2 \leq N, n_1 \neq n_2 \quad (18)$$

$$f_{n_1, a_1, n_2, a_2} \leq x_{n_2, a_2} \quad \text{for all } a_1, a_2 \in A, 1 \leq n_1, n_2 \leq N, n_1 \neq n_2 \quad (19)$$

$$f_{n_1, a_1, n_2, a_2} \geq x_{n_1, a_1} + x_{n_2, a_2} - 1 \quad \text{for all } a_1, a_2 \in A, 1 \leq n_1, n_2 \leq N, n_1 \neq n_2 \quad (20)$$

$$d_{k_1, n_1} \geq f_{n_1, a_1, n_2, a_2} o_{a_1, a_2} \quad \text{for all } a_1, a_2 \in A, 1 \leq n_1, n_2 \leq N, n_1 \neq n_2 \quad (21)$$

$$t \geq \sum_{a \in A} c_a x_{n,a} \quad \text{for all } 1 \leq n \leq N \quad (22)$$

$$x_{n,a} \in \{0, 1\} \quad \text{for all } a \in A, 1 \leq n \leq N \quad (23)$$

$$y_a \geq 0 \quad \text{for all } a \in A \quad (24)$$

$$0 \leq f_{n_1, a_1, n_2, a_2} \leq 1 \quad \text{for all } a_1, a_2 \in A, 1 \leq n_1, n_2 \leq N, n_1 \neq n_2 \quad (25)$$

$$d_{n,a} \geq 0 \quad \text{for all } a \in A, 1 \leq n \leq N \quad (26)$$

$$(27)$$

Implementation

```
[30]: using JuMP
function getSimultaneousDiverseShortPathTradeoff(g::SparseGraph, from::
    AbstractNode, to::AbstractNode, numPaths=3;
    tradeOffWeight=1::Number,
    LInfWeight=0::Number,
    shortestPathTolerance=1::Number,
    silent=true)::Vector{Vector{Edge}}

function supply(n::AbstractNode)
    if (n == from)
        return 1
    elseif (n == to)
        return -1
    end
end
```

```

        else
            return 0
        end
    end
end

edgeCost = Dict()
#Get edge to edge costs
for e1 in g.edges
    #Get hop distance from e1
    dist = getMinDistanceFromUsed(g, Set([e1]))

    #Calculate cost from distance
    ec2 = Dict([e1, e2] => 1 / sqrt(get(dist, e2, 0) + 1) for e2 in g.edges)

    #Add new entries to the edge costs
    merge!(edgeCost, ec2)
end

cost = Dict(e => e[:cost] for e in g.edges)

m = Model(HiGHS.Optimizer)

@variable(m, shortFlow[g.edges] >= 0)
@variable(m, shortFlowSize)

@variable(m, 0 <= flow[1:numPaths, g.edges] <= 1, Int)
@variable(m, tInfy)
@variable(m, 0 <= bothFlow[1:numPaths, g.edges, 1:numPaths, g.edges] <= 1)
@variable(m, diversityCost[1:numPaths, g.edges] >= 0)

#Minimize cost of all paths, make sure we get a shortest path, add
↳tradeoffs for dissimilarity and regularization
    @objective(m, Min, sum(flow[k, e] * e[:cost] for k in 1:numPaths, e in g.
↳edges)
        + (1 + tradeOffWeight + LInfWeight + sum(e[:cost] for e
↳in g.edges)) * sum(shortFlow[e] * e[:cost] for e in g.edges)
        + tradeOffWeight * sum([diversityCost[k1, e1] for k1 in
↳1:numPaths, e1 in g.edges])
        + LInfWeight * tInfy
    )

#Flow constraints for shortest path
@constraint(m, flowConstrShort[n in g.nodes], sum(shortFlow[e] for e in g.
↳fromEdges[n]) == sum(shortFlow[e] for e in g.toEdges[n]) + supply(n))

```

```

    #Get shortest path to add tolerance
    @constraint(m, shortFlowSize == sum(shortFlow[e] * e[:cost] for e in g.
↳edges))

    #Flow constraints for diverse paths
    for k in 1:numPaths
        @constraint(m, [n in g.nodes],
            sum(flow[k, e] for e in g.fromEdges[n]) == sum(flow[k, e] for e in_
↳g.toEdges[n]) + supply(n))
        if (k == 1)
            #make sure at least one path is within tolerance of the shortest_
↳path
            @constraint(m, sum(flow[k, e] * e[:cost] for e in g.edges) <=_
↳shortFlowSize * shortestPathTolerance)
        end
    end

    #Diversity Cost: each edge cost is base on the minimum distance (max_
↳diversity cost) between the edge, and the edge from any other flow

    for k1 in 1:numPaths
        #Setup
        otherPaths = [x for x in 1:numPaths if x != k1]
        @constraint(m, [k2 in otherPaths, e1 in g.edges, e2 in g.edges],_
↳bothFlow[k1, e1, k2, e2] <= flow[k1, e1])
        @constraint(m, [k2 in otherPaths, e1 in g.edges, e2 in g.edges],_
↳bothFlow[k1, e1, k2, e2] <= flow[k2, e2])
        @constraint(m, [k2 in otherPaths, e1 in g.edges, e2 in g.edges],_
↳bothFlow[k1, e1, k2, e2] >= flow[k1, e1] + flow[k2, e2] - 1)

        #Cost is the max cost between edges in path
        @constraint(m, [k2 in otherPaths, e1 in g.edges, e2 in g.edges],_
↳diversityCost[k1, e1] >= bothFlow[k1, e1, k2, e2] * edgeCost[[e1, e2]])
    end

    #Regularization
    if (LInfWeight != 0)
        @constraint(m, [k in 1:numPaths], tInfty >= sum(flow[k, e] * e[:cost]_
↳for e in g.edges))
    else
        @constraint(m, [k in 1:numPaths], tInfty == 0)
    end

    if (silent)
        set_silent(m)
    end

```

```

end

optimize!(m)

if (is_solved_and_feasible(m))
    return [[e for e in g.edges if value(flow[k, e]) > 0.00001] for k in 1:
↪ numPaths]
else
    return Edge[]
end
end
;

```

This formulation runs much more slowly since it is a mixed-integer program and because we need to generate our dissimilarity measure for all pairs of edges. So we are testing on a much smaller 3×3 grid. We're still able to get interesting results, but it shows just how much longer it takes to solve.

```

[31]: #Much slower, need smaller grid
s = 3
Random.seed!(4224)
simulGrid = createGrid(s, s, propGenEdge=genGridCost)
simulWeights = [0, 1, 2, 4]

iterPaths = [getDiversePaths(simulGrid, NodeKey("1,1"), NodeKey("$s,$s"), 3,
↪ tradeOffWeight=k, diversityCost=myDiversityCost,) for k in simulWeights]

# go to opposite corners
simulPaths = [getSimultaneousDiverseShortPathTradeoff(simulGrid, NodeKey("1,1"),
↪ NodeKey("$s,$s"), 3, tradeOffWeight=k) for k in simulWeights]

# do regularization
simulRegPaths = [getSimultaneousDiverseShortPathTradeoff(simulGrid,
↪ NodeKey("1,1"), NodeKey("$s,$s"), 3, tradeOffWeight=4, LInfWeight=k,
↪ shortestPathTolerance=1.1) for k in simulWeights]
;

```

```

[32]: iterPathCost = [getAvgPathCost(pathList) for pathList in simulPaths]
iterMaxPathDifference = [getMaxPathDifference(pathList) for pathList in
↪ simulPaths]
iterPathOverlap = [pathOverlap(ps) for ps in simulPaths]
iterPathDist = [getAvgHopsFromOtherPaths(simulGrid, ps) for ps in simulPaths]

simulPathCost = [getAvgPathCost(pathList) for pathList in simulPaths]
simulMaxPathDifference = [getMaxPathDifference(pathList) for pathList in
↪ simulPaths]
simulPathOverlap = [pathOverlap(ps) for ps in simulPaths]

```

```
simulPathDist = [getAvgHopsFromOtherPaths(simulGrid, ps) for ps in simulPaths]
;
```

Results

No Regularization We first compute iterative and simultaneous models using the same parameters to compare the two. We compute the same statistics we did for the baseline model. Because the grid is so small, we do not calculate statistics on multiple numbers of diverse paths.

Unfortunately, there appears to be no difference between the simultaneous and iterative approaches in this case. It could be that the small size of the graph makes it hard to find bad iterative choices. But this does help justify using the baseline model since it may be that for graph sizes that we can easily solve simultaneously there isn't a significant difference between the two anyway.

Even though are graphs have lines between them, we make the non-solid to show that not every point between the lines are necessarily solutions since these are integer programs. However, we keep the lines to better show trends as well and overlap since there is not much difference in some cases.

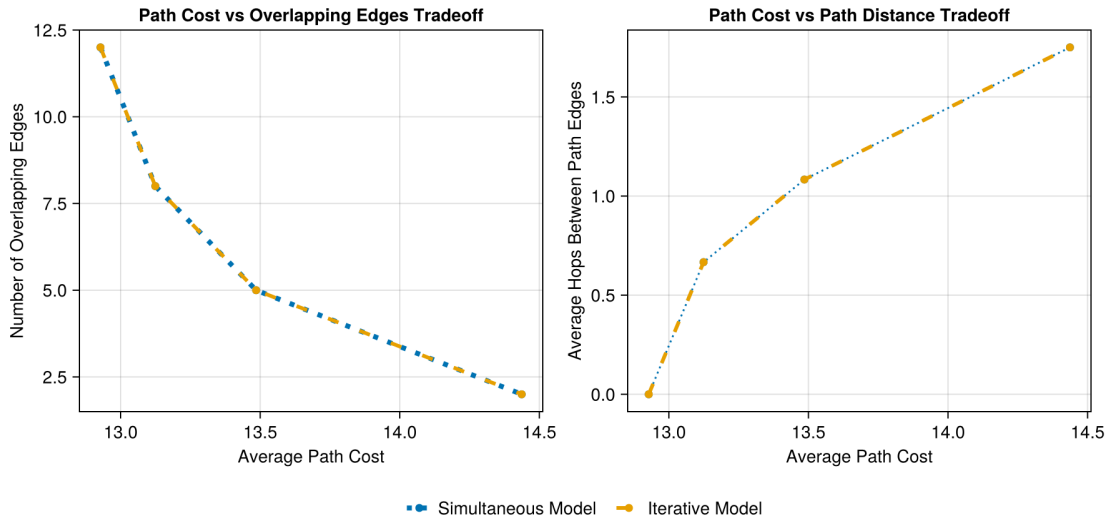
```
[33]: using CairoMakie

f = Figure(size=(900, 450))
ax1 = Axis(f[1, 1], xlabel="Average Path Cost", ylabel="Number of Overlapping_
↳Edges",
           title="Path Cost vs Overlapping Edges Tradeoff")
scatterlines!(ax1, simulPathCost, simulPathOverlap, linewidth=4, linestyle=:
↳dot, label="Simultaneous Model")
scatterlines!(ax1, iterPathCost, iterPathOverlap, linewidth=3, linestyle=(
↳dash, :loose), label="Iterative Model")

ax2 = Axis(f[1, 2], xlabel="Average Path Cost", ylabel="Average Hops Between_
↳Path Edges",
           title="Path Cost vs Path Distance Tradeoff")
scatterlines!(ax2, simulPathCost, simulPathDist, linestyle=:dot)
scatterlines!(ax2, iterPathCost, iterPathDist, linewidth=3, linestyle=(
↳dash, :
↳loose))

Legend(f[2, 1:2], ax1, framevisible=false, orientation=:horizontal)
f
```

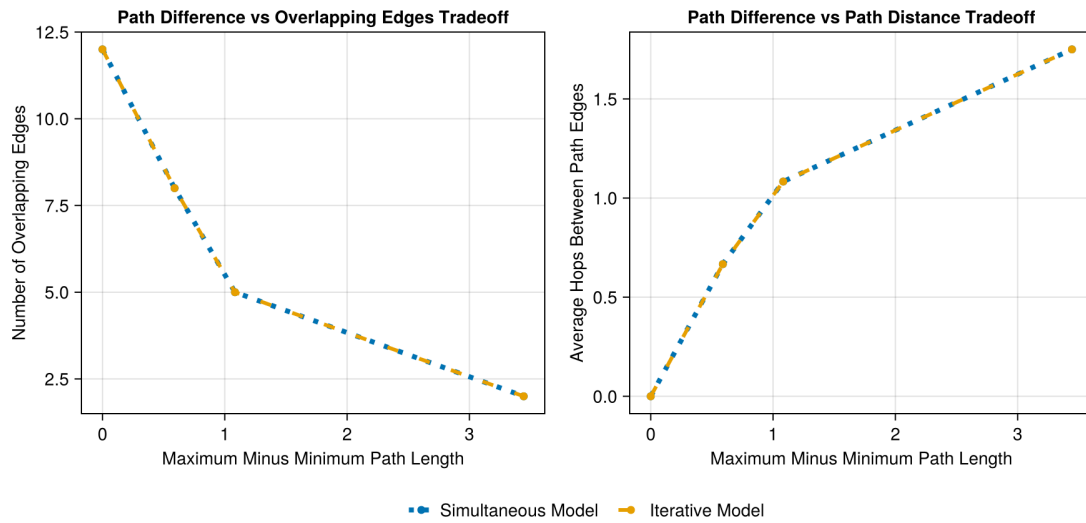
[33]:



```
[34]: f = Figure(size=(900, 450))
ax1 = Axis(f[1, 1], xlabel="Maximum Minus Minimum Path Length", ylabel="Number_
↳ of Overlapping Edges",
title="Path Difference vs Overlapping Edges Tradeoff")
scatterlines!(ax1, simulMaxPathDifference, simulPathOverlap, linewidth=4,
↳ linestyle=:dot, label="Simultaneous Model")
scatterlines!(ax1, iterMaxPathDifference, iterPathOverlap, linewidth=3,
↳ linestyle=(:dash, :loose), label="Iterative Model")

ax2 = Axis(f[1, 2], xlabel="Maximum Minus Minimum Path Length", ylabel="Average_
↳ Hops Between Path Edges",
title="Path Difference vs Path Distance Tradeoff")
scatterlines!(ax2, simulMaxPathDifference, simulPathDist, linewidth=4,
↳ linestyle=:dot)
scatterlines!(ax2, iterMaxPathDifference, iterPathDist, linewidth=3,
↳ linestyle=(:dash, :loose))
Legend(f[2, 1:2], ax1, framevisible=false, orientation=:horizontal)
f
```

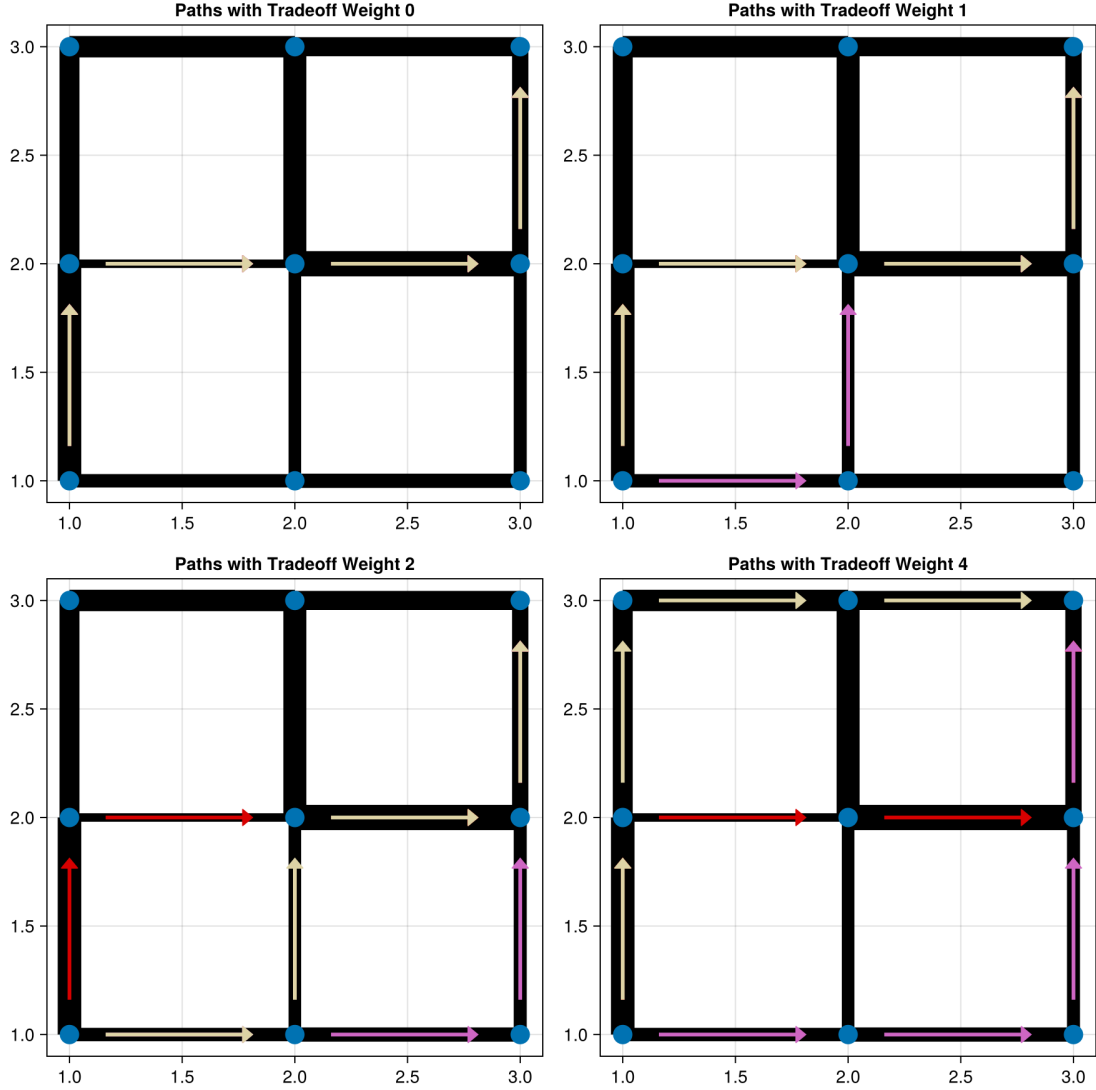
[34]:



```
[35]: f = Figure(size=(900, 900))

for k in 1:length(simulPaths)
    adjK = k - 1
    tradeoffWeight = simulWeights[k]
    ax = Axis(f[adjK÷2+1, adjK%2+1], title="Paths with Tradeoff Weight_
↪$tradeoffWeight")
    plotPaths(ax, simulGrid, simulPaths[k])
end
f
```

[35]:



Regularization We also ran it with a fixed tradeoff and added the L_∞ regularization. We chose a tradeoff value that tends toward diversity so we can maximize the effect of the regularization.

```
[36]: simulRegPathCost = [getAvgPathCost(pathList) for pathList in simulRegPaths]
simulRegMaxPathDifference = [getMaxPathDifference(pathList) for pathList in
    ↪ simulRegPaths]
simulRegPathOverlap = [pathOverlap(ps) for ps in simulRegPaths]
simulRegPathDist = [getAvgHopsFromOtherPaths(simulGrid, ps) for ps in
    ↪ simulRegPaths]
;
```

There is a correlation between increasing L_∞ regularization and lower path costs, but it's not as smooth a shape as when we change the path cost/diversity tradeoff.

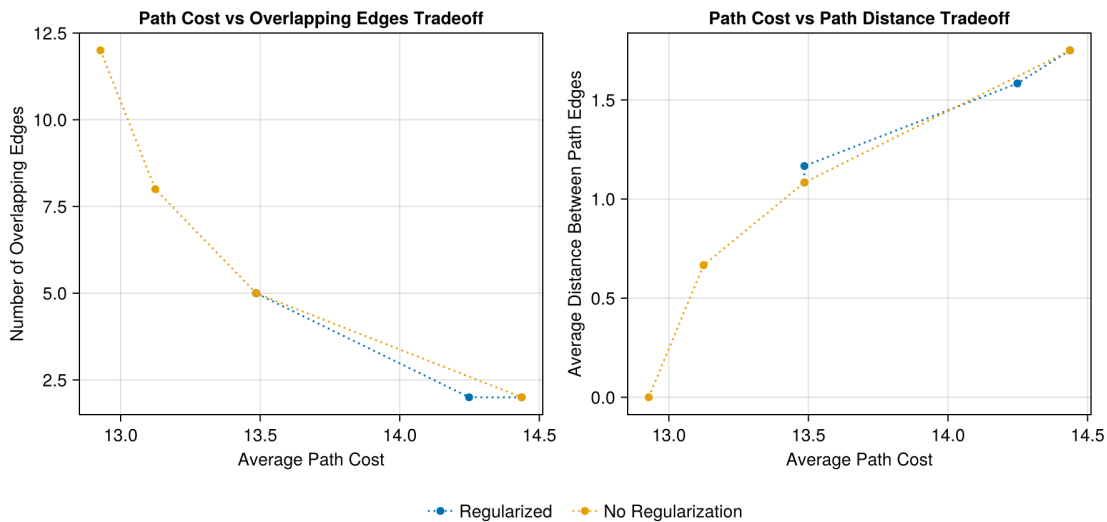
```
[37]: using CairoMakie

f = Figure(size=(900, 450))
ax1 = Axis(f[1, 1], xlabel="Average Path Cost", ylabel="Number of Overlapping
Edges",
title="Path Cost vs Overlapping Edges Tradeoff")
scatterlines!(ax1, simulRegPathCost, simulRegPathOverlap, label="Regularized",
linestyle=:dot)
scatterlines!(ax1, simulPathCost, simulPathOverlap, label="No Regularization",
linestyle=:dot)

ax2 = Axis(f[1, 2], xlabel="Average Path Cost", ylabel="Average Distance
Between Path Edges",
title="Path Cost vs Path Distance Tradeoff")
scatterlines!(ax2, simulRegPathCost, simulRegPathDist, linestyle=:dot)
scatterlines!(ax2, simulPathCost, simulPathDist, linestyle=:dot)

Legend(f[2, 1:2], ax1, framevisible=false, orientation=:horizontal)
f
```

[37]:



As we'd hope, the regularization seems to have a stronger effect on the maximum difference between path lengths, the slope of the pareto curve looks much stronger than when regularizing diversity vs path cost. It's essentially a straight line to the minimum possible. However, it seems like this only has a significant effect vs. the implicit L_1 regularization of the normal objective in the medium-high regularization weights.

```
[38]: f = Figure(size=(900, 450))
ax1 = Axis(f[1, 1], xlabel="Maximum Minus Minimum Path Length", ylabel="Number
of Overlapping Edges",
```

```

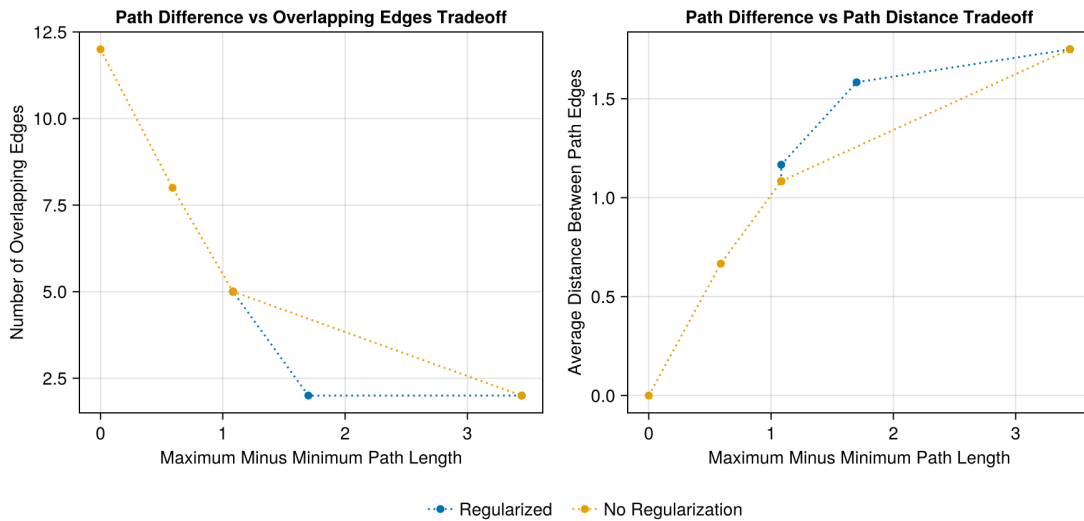
    title="Path Difference vs Overlapping Edges Tradeoff")
scatterlines!(ax1, simulRegMaxPathDifference, simulRegPathOverlap,
    label="Regularized", linestyle=:dot)
scatterlines!(ax1, simulMaxPathDifference, simulPathOverlap, label="No
    Regularization", linestyle=:dot)

ax2 = Axis(f[1, 2], xlabel="Maximum Minus Minimum Path Length", ylabel="Average
    Distance Between Path Edges",
    title="Path Difference vs Path Distance Tradeoff")
scatterlines!(ax2, simulRegMaxPathDifference, simulRegPathDist, linestyle=:dot)
scatterlines!(ax2, simulMaxPathDifference, simulPathDist, linestyle=:dot)

Legend(f[2, 1:2], ax1, framevisible=false, orientation=:horizontal)
f

```

[38]:



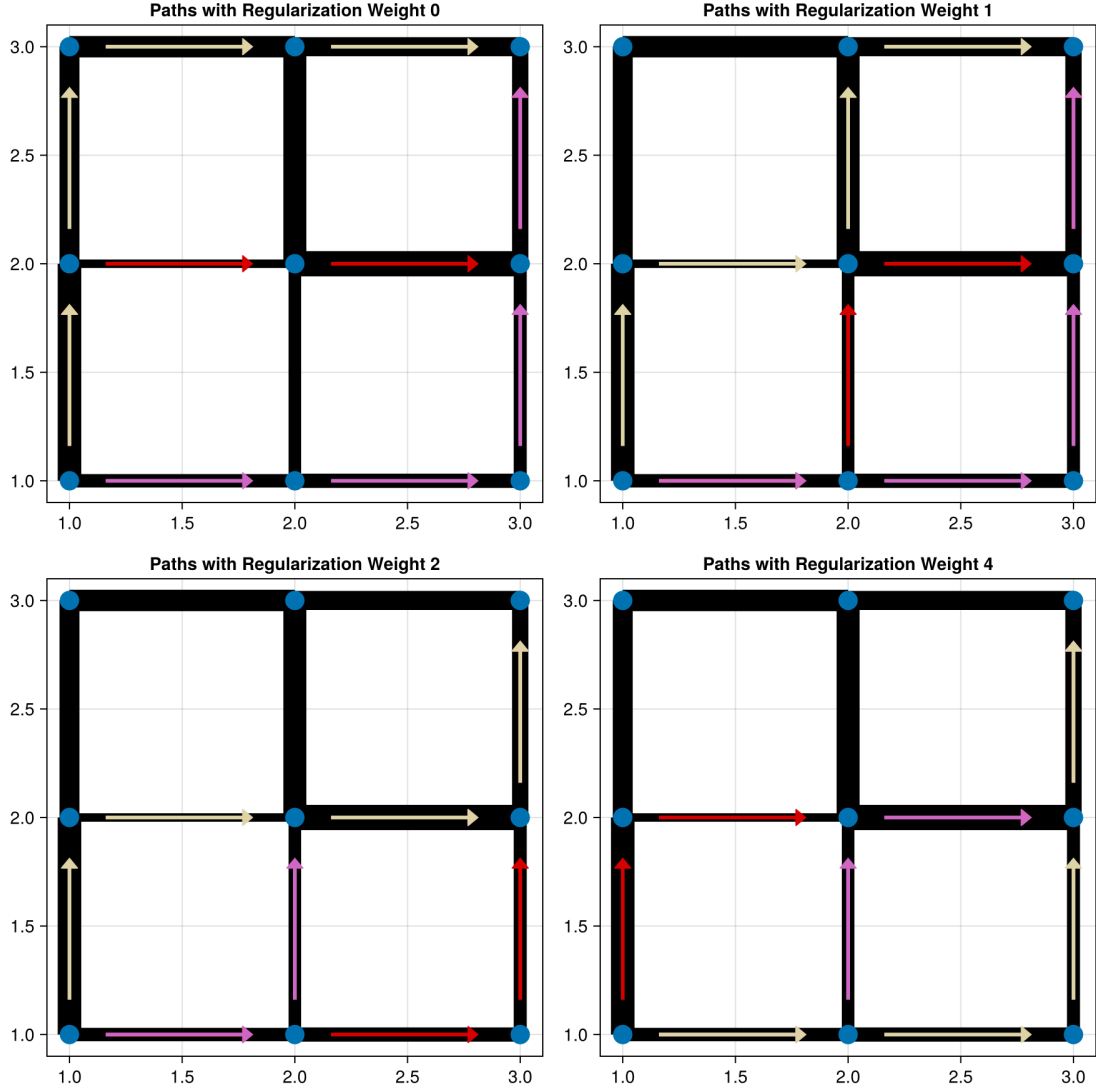
```

[39]: f = Figure(size=(900, 900))

for k in 1:length(simulRegPaths)
    adjK = k - 1
    regularizationWeight = simulWeights[k]
    ax = Axis(f[adjK÷2+1, adjK%2+1], title="Paths with Regularization Weight
        regularizationWeight")
    plotPaths(ax, simulGrid, simulRegPaths[k])
end
f

```

[39]:



Excursion 3: Variance in Arc Costs

In this excursion, we examine how the range of arc costs affects the diverse paths produced. To do so, we draw edge costs from a Normal distribution and change the variance σ^2 .

Excursion 3 Overview

1. Here, we include the code used for this excursion. We define a (seeded) random function to compute edge costs from a normal distribution and scale the variance. We define a function to lay out several subplots of a figure.
2. Here, we show and analyze several plots. We test values $\sigma^2 = 0, 0.5, 1, 5, 25, 100, 500$ and identify main trends in the diverse path plots at different tradeoff weights λ as σ^2 increases. Specifically, we look at:
 1. At which values of λ the paths overlap somewhat (share some but not all edges)

2. At which values of λ the paths intersect (cross over each other) but do not overlap
3. How quickly (wrt increase in λ) the paths approach the corners of the grid
3. We provide a summary of the main trends in the plots.

1. Code We want to assign costs randomly according to a Normal distribution, but don't want any negative costs. When some of the edges have negative costs, the shortest path algorithm is incentivized to add more edges to lower the cost of the path, and this disrupts the construction of diverse paths.

For our 100x100 grid, we generate 10,000 edge costs. This quick assessment ensures that shifting the Normal distribution by 7 results in the first 10,000 numbers after setting the random seed being positive.

```
[40]: using Random

Random.seed!(4224)
for i in range(1,10000)
    x = randn()+7
    if x < 0
        println("uh oh")
    end
end
```

Then, we define a function to randomly generate costs from a Normal distribution, then scale to have the desired variance. We multiply by the square root of variance, $\sqrt{\sigma^2} = \sigma$, also known as standard deviation. This is simply because when we scale a distribution by a factor of c , variance (which is nonlinear) is scaled by c^2 .

```
[41]: function generateGridCostWithVarianceFunc(variance)
    Random.seed!(4224) # reset the random seed inside the function because we
    ↪ know the first 10,000 randn() numbers are > -7
    return (to,from) -> Dict{:cost} => (sqrt(variance) * (randn() + 7)))
end
```

[41]: generateGridCostWithVarianceFunc (generic function with 1 method)

Here we define a plotting function that takes in Axis as a parameter, so that we can create subplots within a figure. This helps us visualize the diverse path plots across several different tradeoff weights without having to scroll through an output cell.

```
[44]: function plotPathsAx(ax::Axis, g::SparseGraph, paths::
    ↪ Vector{Vector{Edge}}=Vector{Edge}[])
    edgesInPath = length(paths) == 0 ? Set() : Set(reduce(vcat, paths))
    edgesNotInPath = [e for e in g.edges if !in(e, edgesInPath)]
    dispEdges = sort(g.edges, by=x -> in(x, edgesInPath) ? 1 : 0) #paint path
    ↪ edges last
    colors = [in(e, edgesInPath) ? :blue : :black for e in g.edges]

    linesegments!(ax,
```

```

        reduce(vcat, [[e.from.x, e.to.x] for e in g.edges]),
        reduce(vcat, [[e.from.y, e.to.y] for e in g.edges]),
        color=:black, linewidth=(reduce(vcat, [[e[:cost], e[:cost]] for e in g.
↪edges])) .- 0.5) .^ 2)

    if (length(g.edges) < 1000)
        scatter!(ax, [n.x for n in g.nodes], [n.y for n in g.nodes],
↪markersize=22)
    end

    for p in 1:length(paths)
        pathEdges = paths[p]
        arrows2d!(ax,
            [e.from.x for e in pathEdges], [e.from.y for e in pathEdges],
            [e.to.x - e.from.x for e in pathEdges], [e.to.y - e.from.y for e in
↪pathEdges],
            color = fill(p, length(pathEdges)),
            colormap = :glasbey_bw_minc_20_minl_30_n256,
            colorrange = (1, max(2, length(paths))),
            align=-0.25, lengthscale=0.65,
            shaftwidth=3
        )
    end

    return f
end
;

```

2. Plots Below, we provide the code to generate 3 diverse paths across tradeoff weights $\lambda = 0, 0.1, 0.5, 2, 8, 16, 256$ for a given value of variance σ^2 . Then, we include and analyze the diverse paths obtained at $\sigma^2 = 0, 0.5, 1, 5, 25, 100, 500$.

```

[46]: # we assume we've loaded all the necessary functions.

using CairoMakie, Random

variance = 0.5 # here is where we set the variance. this is the only line we
↪need to change to generate the different plots.

myGrid = createGrid(100, 100,
↪propGenEdge=generateGridCostWithVarianceFunc(variance))

weights = [0, 0.1, 0.5, 2, 8, 16, 256]

allPaths = [getDiversePaths(myGrid, NodeKey("2,2"), NodeKey("97,97"), 3,
↪diversityCost=myDiversityCost, tradeOffWeight=k) for k in weights]

```

```

pathCost = [getAvgPathCost(pathList) for pathList in allPaths]

pathOver = [pathOverlap(ps) for ps in allPaths]

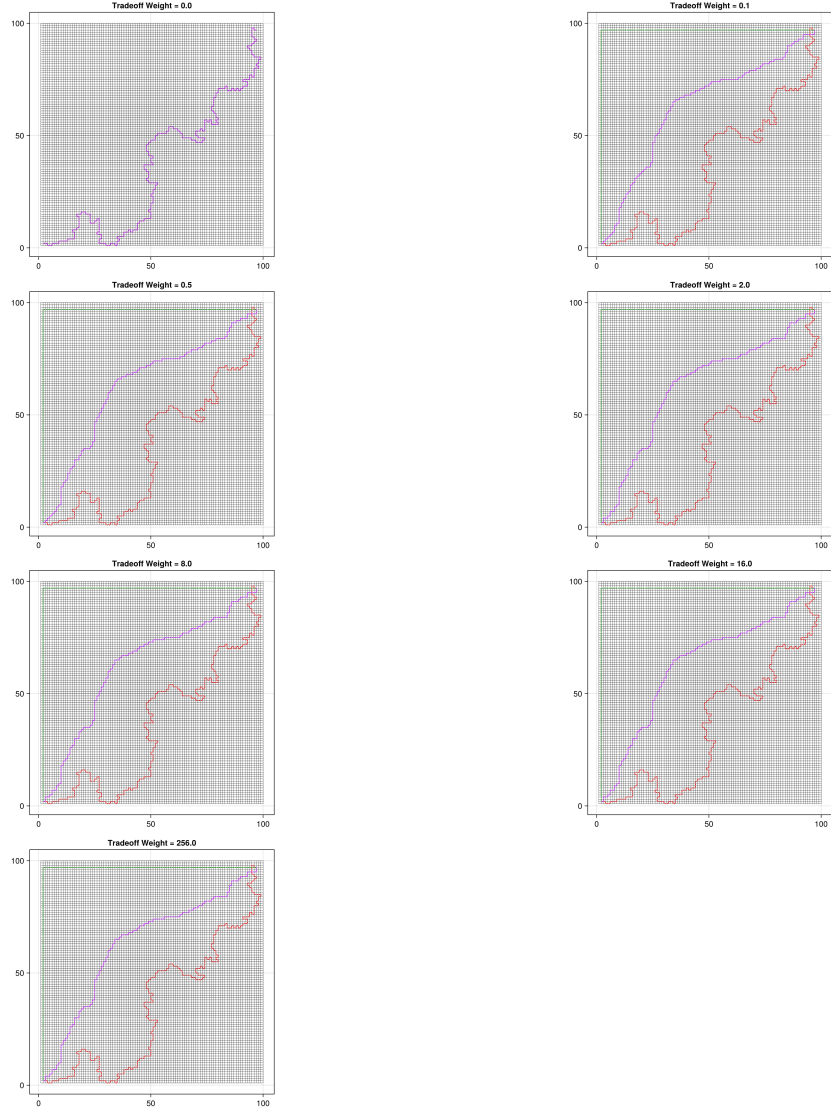
pathDist = [getAvgHopsFromOtherPaths(myGrid, ps) for ps in allPaths]

# create figure
f = Figure()
px_per_plot = 1000
num_cols = 2
num_rows = length(allPaths)%2+1
f = Figure(size = (num_cols * px_per_plot, num_rows * px_per_plot))

# add subplots
for k in 1:length(allPaths) # each path in allPaths corresponds to a tradeoff
    ↪weight value
    adjK = k - 1
    tradeoffWeight = weights[k]
    ax = Axis(f[adjK÷2+1, adjK%2+1], aspect=1, title="Tradeoff Weight = ↪
    ↪$tradeoffWeight")
    plotPathsAx(ax,myGrid, allPaths[k])
end
f
;

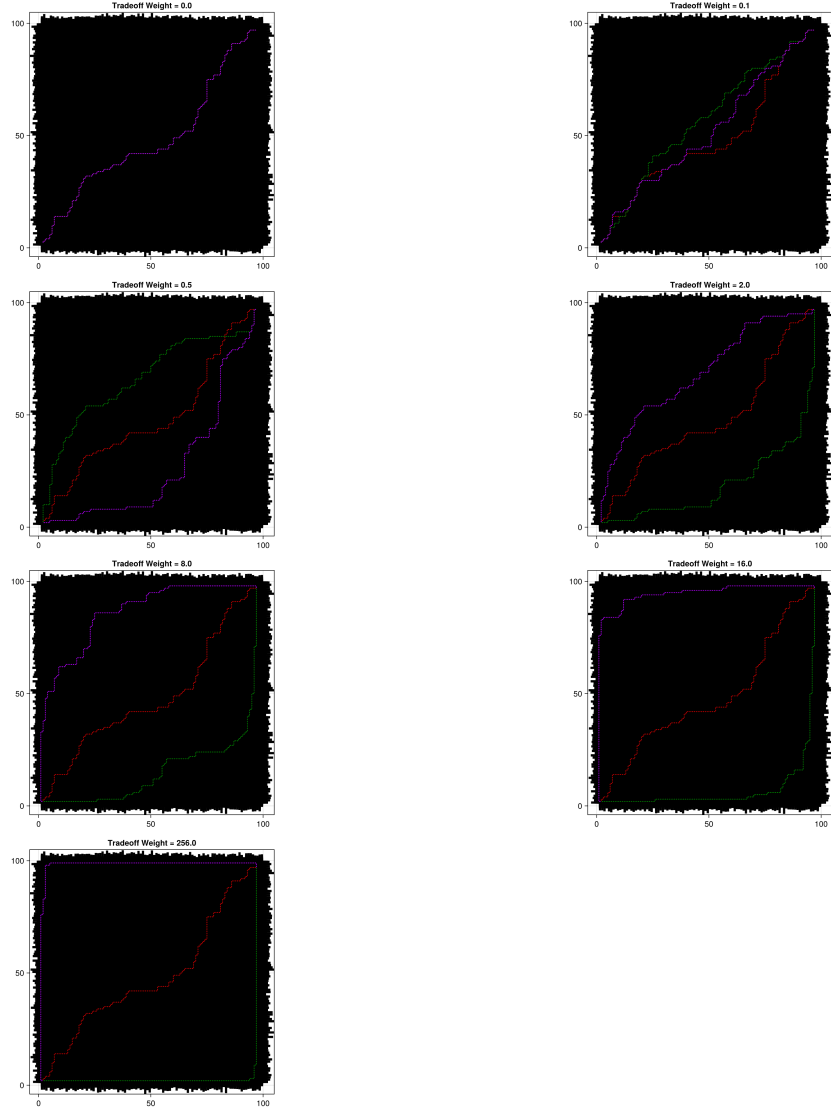
```

Variance = 0



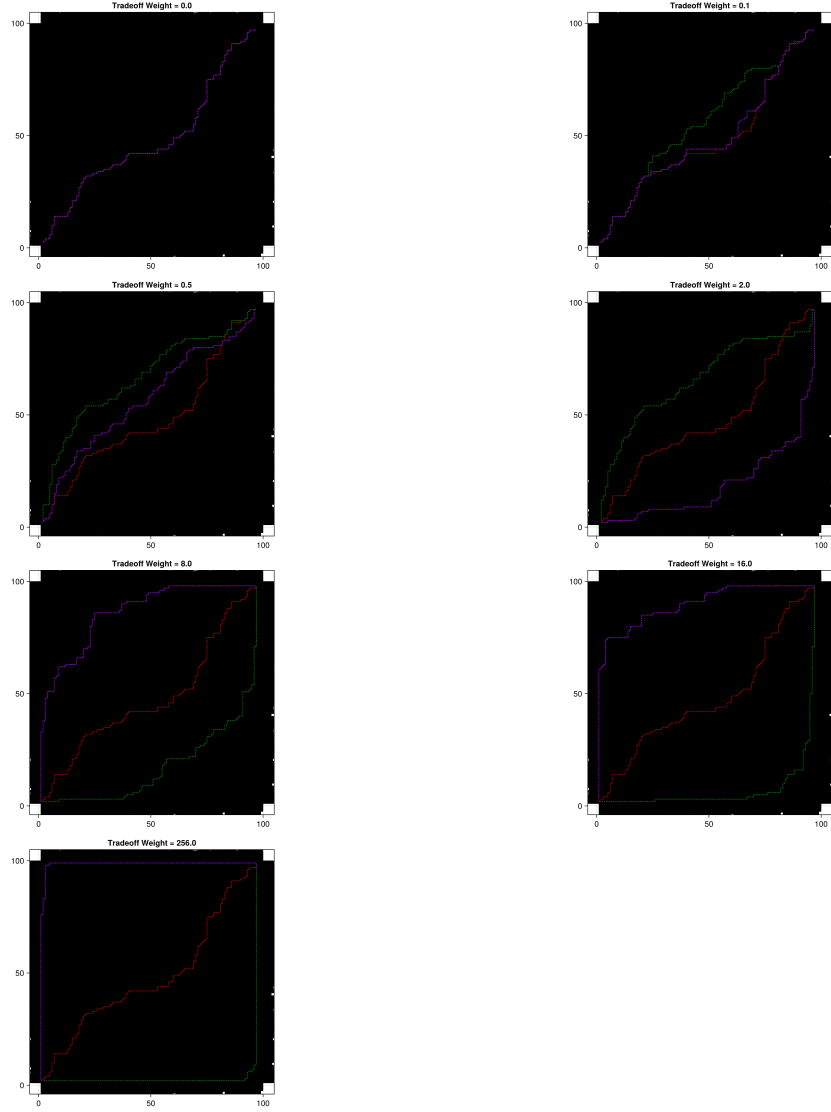
When $\text{variance} = 0$, all costs are equal to 7 by construction, so the algorithm simply finds the shortest path, and then shortest diverse paths. We see that increasing the λ doesn't change the chosen paths once $\lambda > 0$. The paths intersect near the top right corner, but only overlap in the case where $\lambda = 0$ and all paths are identical.

Variance = 0.5



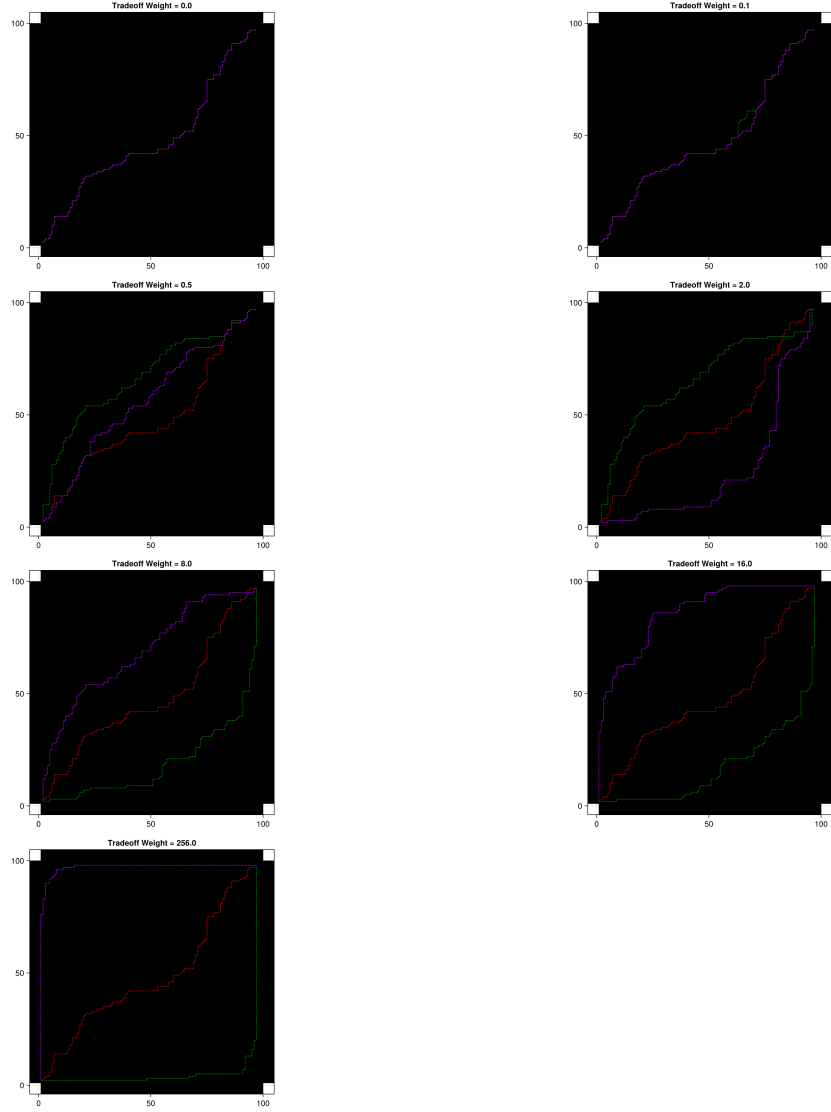
When we increase to $\sigma^2 = 0.5$, we see that the paths now actually change as the λ increases. The paths are again identical at $\lambda = 0$, overlap somewhat at $\lambda = 0.1$, and intersect at $\lambda = 0.5, 2$. For $\lambda \geq 8$, the paths mostly steer clear of each other. At $\lambda = 256$, the green and purple paths are almost against the corners of the grid.

Variance = 1



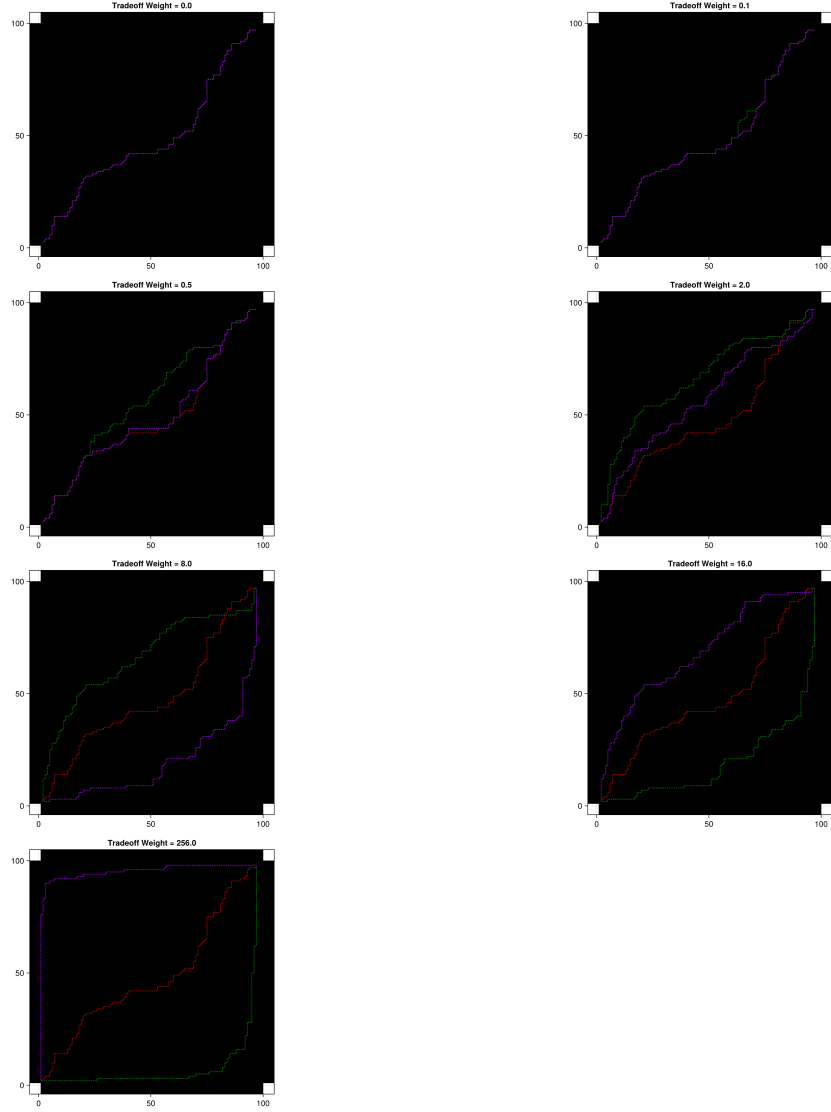
We see that compared to the plots where variance $\sigma^2 = 0.5$, the diverse paths approach the corners slightly more slowly when $\sigma^2 = 1$. The paths overlap somewhat at tradeoff $\lambda = 0.1, 0.5$, and intersect at $\lambda = 2$.

Variance = 5



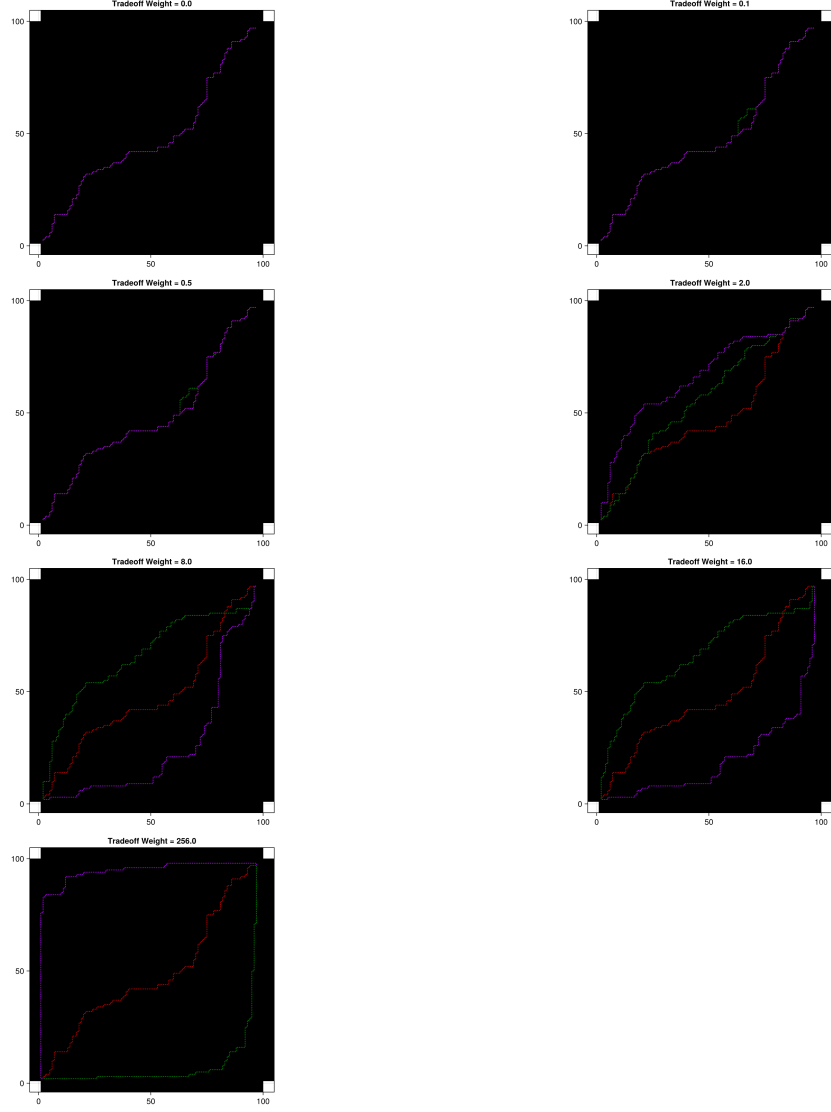
At $\lambda = 0.1$, the red and purple paths are identical, and the green path mostly overlaps them. The green and purple paths have minor overlap at $\lambda = 2$, and intersect at $\lambda = 8$. We notice they approach the corners of the grid more slowly than when $\sigma^2 = 1$; when $\sigma^2 = 5$, the corners are more rounded as they approach the edges of the grid.

Variance = 25



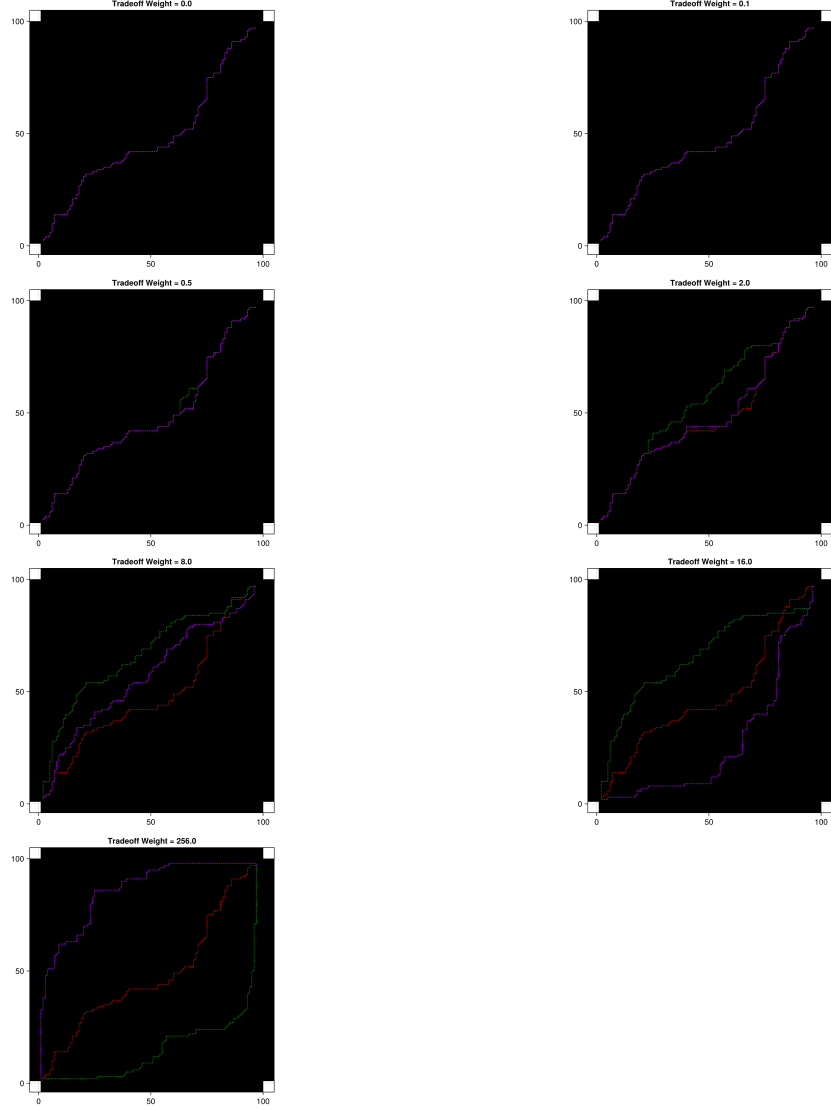
The $\lambda = 0.1$ plot looks identical to the case where $\sigma^2 = 5$, and the $\lambda = 0.5$ has higher overlap. The paths intersect at $\lambda = 16$, and seem to approach the corners of the grid even more slowly than when $\sigma^2 = 5$; the green and purple paths are even closer to the center of the grid.

Variance = 100



On the $\sigma^2 = 100$ plots, we see a clear continuation of the earlier trends: the paths have a higher degree of overlap at higher values of λ , and spread out more slowly. Here, the $\lambda = 0.1$ and 0.5 plots look identical, having total overlap of the red and purple paths, with the green path mostly overlapping them. The paths continue to intersect at $\lambda = 16$, and are farther in from the corners at $\lambda = 256$ than in the $\sigma^2 = 25$ case.

Variance = 500



The $\sigma^2 = 500$ case again continues the trends we've identified. The $\lambda = 2$ plot looks identical to the $\sigma^2 = 25, \lambda = 0.5$ plot, demonstrating that as variance increases, overlap continues into higher values of λ . On the above $\lambda = 256$ plot, the green and purple paths look about as close to the red path as they are to the corners of the grid.

Excursion 3 Summary To summarize, we know that at a constant, nonzero variance, the diverse paths progress through the following phases as we increase the tradeoff weight λ : 1. Identical: At $\lambda = 0$, the paths overlap completely. 2. Overlap: Then, at low values of λ , paths may overlap somewhat but not completely. 3. Intersect: As we increase λ further, paths cease to overlap, but may cross over once or twice, in places. These places approach the corners as λ increases. 4. Steer Clear: At high values of λ , the paths avoid any intersection.

As we increase variance σ^2 , these above phases shift according the following trends: 1. Identical: Regardless of variance, the paths overlap completely when $\lambda = 0$. 2. Overlap: The paths have greater overlap, and the overlap phase continues to higher values of λ . 3. Intersect: The intersect

phase also continues to higher values of λ , and intersections move toward the corners more slowly as λ increases. 4. Steer Clear: This phase is delayed by the prolonging of phases 2 and 3. Even once the paths no longer intersect, they are closer together than at lower values of σ^2 ; they approach the edges of the grid more slowly.

Overall, as variance increases, the paths grow apart more slowly. The greater range of costs likely incentivizes the diverse path algorithm to prioritize the path with the fewest edges to avoid high cost edges, making the paths less similar. This establishes that not only λ but also σ^2 influences whether cost or diversity cost of a path is prioritized. A value of λ has less influence on a graph with high variance in edge costs than low variance in edge costs.

6. Conclusion

We have presented an iterative process of solving linear programs to find k diverse and short paths. Utilizing tradeoffs and total unimodularity we can quickly provide alternative routes for users to choose from, and can allow flexibility on how important speed is versus diversity. Even a mild weight towards diversity can give us much more diverse paths without sacrificing too much path cost.

There is a way to solve the problem with a single model, but it is slow and only feasible for the smallest instances of this problem. However, it does allow for L_∞ regularization.

Since diversity is such a fuzzy topic, there’s multitudes of ways we could improve upon this. Many similar problems utilize dynamic programming in their approaches. In addition, we could investigate the human factors of diversity, perhaps taking into account neighborhoods, scenery, traffic patterns and the like to provide diverse paths that are not just far from each other, but also have a more subjective “feeling” of being different.

Lastly, there are algorithmic improvements that could be made as well. We have already done some initial experiments into an algorithm based on Expectation Maximization algorithms, where we find a new set of diverse paths using diversity costs from a prior candidate set of diverse paths, seemed to give us speed that makes calculating on large graphs feasible while also giving us better results and regularization, but we did not include that for time and space reasons.

References

- [1] Ali Moghanni, Marta Pascoal, and Maria Teresa Godinho. Finding shortest and dissimilar paths. *International Transactions in Operational Research*, 29(3):1573–1601, 2022.
- [2] Lingxiao Li, Muhammad Aamir Cheema, Hua Lu, Mohammed Eunus Ali, and Adel N. Toosi. Comparing alternative route planning techniques: A comparative user study on melbourne, dhaka and copenhagen road networks. *IEEE Transactions on Knowledge and Data Engineering*, 34(11):5552–5557, 2022.
- [3] Conforti, M., Cornuéjols, G., and Zambelli, G. (2014). *Integer Programming*. Graduate Texts in Mathematics, Vol. 271. Springer, Cham.